MASTER REFERENCE · 2026 EDITION

SQL Data Types

From basic to advanced — the complete guide

Every type family explained from first principles to production edge cases: storage, range, precision, implicit conversion, indexing cost and the exact syntax for the five engines analysts meet most.

MySQL 8 | PostgreSQL 16 | SQL Server | Oracle | SQLite

PREPARED BY

Niraj Laxman Jadhav

Data Analyst · Excel · SQL · Power BI · Python

Version 1.0

October 2026

How to use this guide

This handbook is written for people who have to **choose** a data type and then live with that choice: analysts building tables, engineers reviewing schemas, and anyone who has ever wondered why a total came out as 0.30000000000000004. It runs from “what is a type?” to production edge cases, in the order you meet them.

Read it in order Parts 1–5 build the mental model: foundations, numbers, text, dates, then boolean, binary and identifiers. If you are new to SQL, this is the path.	**Read it as a reference** Parts 6–8 are lookup material: structured and special types, casting and storage mechanics, then the cross-engine mapping tables. Jump straight to the engine you use.
Use it before a design review Part 9 is a checklist, a cheat sheet, 15 exercises with answers and a glossary. Print the cheat sheet and keep it beside your keyboard.	**Follow the engine tags** MySQL 8, PostgreSQL 16, SQL Server 2022, Oracle 23ai and SQLite 3 are all covered. Where they disagree, the disagreement is the lesson.

Conventions used in the examples

- Code blocks are real, runnable SQL. The small caption above each block names the engine when the syntax is engine-specific; otherwise the statement works on all five engines.
- Keywords are blue, type names teal, string literals amber, comments grey — so you can see at a glance which part of a declaration is the type.
- **NOTE** marks context you need once. **TIP** marks a shortcut. **WARNING** marks data loss. **COMMON TRAP** marks the mistake that shows up again and again in real schemas. **BEST PRACTICE** is the house rule to adopt. **GO DEEPER** is the advanced detail.
- Storage sizes are the declared sizes from each vendor's documentation. Real on-disk cost adds row headers, NULL bitmaps, alignment padding and index overhead — Part 7 shows how to estimate it.
- Ranges are inclusive at both ends unless stated. ± means a symmetric signed range. “n” always means the declared length or precision.

The five engines at a glance

Engine	Typing model	Signature strength	Signature surprise
MySQL 8	Static, per column	DECIMAL, TINYINT UNSIGNED, ENUM, JSON	No real BOOLEAN; INT(11) is a width, not a size
PostgreSQL 16	Static, strongly enforced	UUID, JSONB, ARRAY, RANGE, custom types	VARCHAR(n) has no performance benefit over TEXT
SQL Server 2022	Static, implicit-cast friendly	DATETIME2, NVARCHAR, MONEY, hierarchyid	Implicit conversion can silently kill an index
Oracle 23ai	Static, NUMBER-centric	NUMBER(p,s), TIMESTAMP WITH TIME ZONE, VECTOR	DATE includes a time but no fractional seconds
SQLite 3	Dynamic, five affinities	Zero-config, one file, STRICT tables	A column can hold any type; no DATE type at all

Table 0.1 — Knowing which model you are in explains most “weird” behaviour.

TIP · One sentence to remember
A data type buys you correctness at the cost of constraints — the whole art is choosing constraints that describe your data honestly.

Contents

Nine parts · foundations to production mechanics

Part 1 · Foundations of Data Types . . . 5

1.1 What a data type actually decides . . . 5
1.2 Why types matter more in analytics than in app code . . . 5
1.3 The universal type families . . . 5
1.4 Anatomy of a declaration . . . 6
1.5 Fixed versus variable length . . . 7
1.6 NULL: not a type, but part of every type . . . 7
1.7 A type does not just cost space — it costs space everywhere . . . 8
1.8 Choosing a type: the decision order . . . 8
1.9 Reading a real schema . . . 8
1.10 Part 1 in one page . . . 9

Part 2 · Numeric Types . . . 10

2.1 The numeric family tree . . . 10
2.2 Integer types: exact by definition . . . 10
2.3 Auto-numbering: AUTO_INCREMENT, IDENTITY, SERIAL . . . 11
2.4 DECIMAL and NUMERIC: exact by declaration . . . 12
2.5 FLOAT, REAL and DOUBLE PRECISION . . . 13
2.6 MONEY types (and why most teams avoid them) . . . 14
2.7 Integer arithmetic and division: the silent zero . . . 14
2.8 Rounding, truncation and the functions you need . . . 15
2.9 BIT and bit masks . . . 15
2.10 Engine cheat sheet: numeric types . . . 15
2.11 Numeric pitfalls checklist . . . 16

Part 3 · Character & String Types . . . 17

3.1 The three text shapes . . . 17
3.2 CHAR versus VARCHAR: the padding problem . . . 17
3.3 Bytes, characters and the utf8mb4 rule . . . 18
3.4 Collation: how text sorts and compares . . . 18
3.5 Length semantics: LENGTH, CHAR_LENGTH and the 65,535 limit . . . 19
3.6 Text search: LIKE, regex and full-text . . . 20
3.7 Practical rules for text columns . . . 20
3.8 Engine cheat sheet: character types . . . 20

Part 4 · Date & Time Types . . . 22

4.1 The four temporal concepts . . . 22
4.2 The temporal type catalogue . . . 22
4.3 Storage cost per engine . . . 23
4.4 Instants versus wall clocks, decided properly . . . 23
4.5 Time zones, DST and the two-hour bug . . . 24
4.6 Ranges, precision and the 2038 problem . . . 24
4.7 Date arithmetic and the functions worth memorising . . . 25
4.8 Generating calendars and bucketing time . . . 26
4.9 Temporal pitfalls checklist . . . 27
4.10 Engine cheat sheet: temporal types . . . 27

Part 5 · Boolean, Binary & Identifiers . . . 28

5.1 BOOLEAN and its four impersonations . . . 28
5.2 Binary types: BINARY, VARBINARY, BLOB, BYTEA, RAW . . . 29

5.3 Identifiers: BIGINT, UUID, ULID and natural keys . . . 30
5.4 Special-purpose types you will meet in schemas . . . 31

Part 6 · Structured & Special Types . . . 32

6.1 When a structured type is the right answer . . . 32
6.2 JSON and JSONB . . . 32
6.3 XML and other document types . . . 34
6.4 ARRAY types . . . 34
6.5 ENUM and SET . . . 35
6.6 RANGE and MULTIRANGE (PostgreSQL) . . . 35
6.7 Spatial types . . . 36
6.8 Vectors and embeddings . . . 36
6.9 User-defined types: domains, composites, extensions . . . 37
6.10 Special-type pitfalls . . . 37

Part 7 · Casting & Storage Mechanics . . . 38

7.1 Implicit and explicit conversion . . . 38
7.2 Sargability: what makes an index usable . . . 39
7.3 Type conversion tables worth knowing . . . 39
7.4 Storage mechanics: how a row is laid out . . . 40
7.5 What a type change costs . . . 41
7.6 Partitioning and clustering follow the type . . . 41
7.7 Case study: a 40× slower report, fixed by two type changes . . . 41

Part 8 · Types Across Engines . . . 43

8.1 Three rules for mapping types . . . 43
8.2 Numeric mapping . . . 43
8.3 Character and binary mapping . . . 44
8.4 Temporal mapping . . . 44
8.5 Identifiers and structured mapping . . . 44
8.6 SQLite: five affinities, no data types . . . 45
8.7 Migration pitfalls by direction . . . 46
8.8 The pre-migration checklist . . . 46

Part 9 · Cheat Sheet & Practice . . . 47

9.1 The one-page cheat sheet . . . 47
9.2 Fifteen exercises . . . 48
9.3 Answers, with reasoning . . . 48
9.4 Glossary . . . 50
9.5 The last checklist . . . 50

PART 1

Foundations of Data Types

What a type is, what it costs, and why the choice is a design decision

Before the catalogue of types, one idea: a data type is a **contract** the database enforces on every value, on every row, forever. Get the contract right and the database protects you. Get it wrong and you spend the next two years writing CAST() around it.

1.1 What a data type actually decides

A type declaration answers four questions at once. Change the type and you change all four, often without noticing.

Decision	What the engine fixes	What breaks if you get it wrong
Encoding	How the bytes are laid out: two's complement for integers, IEEE 754 for floats, a length prefix for strings, an epoch offset for timestamps.	Wrong family = values reinterpreted, e.g. 20240131 stored as an INT cannot be sorted as a date.
Domain	Which values are legal and what happens to illegal ones: reject, round, truncate or wrap.	Silent truncation: 'Niraj Jadhav' into VARCHAR(10) becomes 'Niraj Jad' on MySQL without strict mode.
Operations	Which functions and operators apply: + for numbers, || for strings in some engines, INTERVAL arithmetic for dates.	'2024-01-31' + 1 means different things in different types and engines.
Ordering	How values compare and sort, including collation for text and time zone for timestamps.	A “sorted” report that is alphabetical instead of chronological, or case-sensitive when users expect otherwise.

Table 1.1 — The four decisions inside every type declaration.

1.2 Why types matter more in analytics than in app code

Application code fails loudly. Analytics fails quietly: a division that returns an integer, a timestamp that loses its time zone, a decimal that becomes a float and drifts by a cent at the eighth decimal place. The wrong total reaches a dashboard and someone makes a decision on it. Four reasons the type choice dominates:

- **Cost compounds.** A BIGINT where INT would do costs 2× per column, and every index on that column costs 2× as well. On a billion-row fact table that is tens of gigabytes and a slower scan, forever.
- **Types steer the optimiser.** Comparing two columns of different types can forbid an index seek, turn a 5 ms lookup into a 4-second scan, and nobody notices until the table grows.
- **Types are a contract with other tools.** Power BI, pandas, dbt and every downstream consumer read your declared types. A DATE column is a date in the report; a VARCHAR date is text the user must slice manually.
- **Types encode meaning.** DECIMAL(19,4) says money. TIMESTAMPTZ says “an instant”. BOOLEAN says two states. Those hints survive documentation, staff turnover and migrations.

1.3 The universal type families

Vendors disagree on names, not on concepts. Learn the nine families and every engine becomes a dialect of one language.

Family	What it holds	Typical members	Watch out for
Integer	Whole numbers	TINYINT, SMALLINT, INT, BIGINT, SERIAL, IDENTITY	Overflow, unsigned ranges, integer division
Exact decimal	Money and fixed-point	DECIMAL, NUMERIC, NUMBER(p,s), MONEY	Scale you forgot to declare, rounding mode
Approximate	Scientific values	FLOAT, REAL, DOUBLE PRECISION, BINARY_FLOAT	Binary representation, equality tests, accumulation error
Character	Text and codes	CHAR, VARCHAR(n), TEXT, NVARCHAR, CLOB, VARCHAR2	Bytes vs characters, collation, off-page storage
Temporal	Instants and durations	DATE, TIME, DATETIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL	Time zone loss, fractional seconds, 2038
Boolean	Two states	BOOLEAN, BIT, TINYINT(1), CHAR(1)	Three-valued logic with NULL, engine emulation
Binary	Raw bytes	BINARY, VARBINARY, BLOB, BYTEA, RAW, IMAGE	Encoding, size caps, backups
Structured	Documents inside a column	JSON, JSONB, XML, ARRAY, ENUM, RANGE, HSTORE	Indexability, schema drift, query complexity
Special	Purpose-built	UUID, INET, MACADDR, GEOGRAPHY, TSVECTOR, VECTOR, ROWVERSION	Extension required, portability, index type

Table 1.2 — The nine families used throughout this guide.

NOTE · Names travel badly, families travel well
Oracle NUMBER(11) maps to MySQL DECIMAL(11,0), SQL Server DECIMAL(11,0), PostgreSQL NUMERIC(11,0) and SQLite NUMERIC. The name is not portable; the family and parameters are. That is why Part 8 is a mapping table rather than a list of names.

1.4 Anatomy of a declaration

A column definition has up to five parts. Reading them separately is how you review a schema quickly.

mysql 8 · every part of a column definition

```
-- name type + params modifiers default
-- ──────── ──────────────────── ──────────────────────────────── ────────────────
CREATE TABLE orders (
    order_id     BIGINT UNSIGNED           NOT NULL AUTO_INCREMENT,        -- identity + range
    customer_id  INT UNSIGNED              NOT NULL,                        -- foreign key
    status       ENUM('new','paid','void') NOT NULL DEFAULT 'new',          -- domain constraint
    amount       DECIMAL(12,2)             NOT NULL DEFAULT 0.00,           -- exact money
    currency     CHAR(3)                   NOT NULL DEFAULT 'INR',
    note         VARCHAR(500)              CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci,
    created_at   TIMESTAMP                 NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (order_id),
    INDEX idx_customer (customer_id)
) ENGINE = InnoDB;
```

- **Type + parameters** — the family plus its size/precision: VARCHAR(500), DECIMAL(12,2), DATETIME(3). Parameters are not decoration; they are the contract's dimensions.
- **Numeric modifiers** — UNSIGNED, ZEROFILL (MySQL), WITH TIME ZONE (PostgreSQL/Oracle), SRID (spatial). They change the legal range, not the storage class.
- **Character modifiers** — CHARACTER SET and COLLATE decide how text is stored and compared.
- **Identity modifiers** — AUTO_INCREMENT, IDENTITY(1,1), GENERATED ... AS IDENTITY, SERIAL, sequences. Covered in Part 2.
- **Constraints and defaults** — NOT NULL, DEFAULT, CHECK, UNIQUE, REFERENCES. In practice the constraint set is part of the type's meaning.

1.5 Fixed versus variable length

Three storage strategies appear in every engine: **fixed** (allocate the declared size always), **variable with inline data** (length prefix plus the bytes, up to a threshold) and **variable with off-page overflow** (a pointer stays in the row, the payload moves elsewhere). The strategy decides what a wide column really costs you.

Strategy	Examples	Cost per row	Consequence
Fixed	INT, BIGINT, DATE, CHAR(10), BINARY(16), UNIQUEIDENTIFIER	Always the declared width, plus alignment padding	Predictable and fast; wasted space when values are short
Variable inline	VARCHAR(n), NVARCHAR(n), VARBINARY(n), DECIMAL	2 bytes of length + actual bytes (engine-specific)	Compact; row can grow on update
Variable off-page	TEXT, LONGTEXT, BLOB, VARCHAR(MAX), BYTEA, CLOB	20-byte pointer in the row + payload in overflow pages	Row stays slim; reading the column requires extra I/O

Table 1.3 — The three storage strategies.

GO DEEPER · Off-page is not free
In PostgreSQL, values over ~2 KB are compressed and moved to TOAST tables; a pointer remains. Selecting the column forces a second read. In SQL Server, VARCHAR(MAX) values over 8 KB go to row-overflow pages. A dashboard that runs SELECT * on a table with a CLOB reads gigabytes it never displays — always list columns explicitly.

1.6 NULL: not a type, but part of every type

Any nullable column can hold a fourth kind of value alongside its domain: NULL, meaning unknown or not applicable — not zero and not empty string. NULLs obey three-valued logic: comparisons return TRUE, FALSE or UNKNOWN, and WHERE keeps only TRUE.

universal · three-valued logic in one screen

```
SELECT * FROM customers WHERE city = 'Mumbai';   -- NULL cities excluded
SELECT * FROM customers WHERE city <> 'Mumbai';  -- NULL cities excluded AGAIN
SELECT * FROM customers WHERE city IS NULL;      -- the only reliable test
SELECT COALESCE(city, 'Unknown') FROM customers; -- substitute for display
SELECT COUNT(*), COUNT(city) FROM customers;     -- count all rows vs non-NULL city
```

- **Aggregates skip NULL.** AVG over 10 values where 4 are NULL divides by 6, not 10. This is the single most common source of “wrong” averages in reports.
- **NOT IN with NULL returns nothing.** x NOT IN (1, NULL) is never TRUE. Use NOT EXISTS, or filter NULLs out of the list.
- **NULL is nearly free.** Engines store a NULL bitmap or a null flag; a NULL in a fixed-width column usually saves nothing, but a NULL in a variable column costs one byte.
- **NULLs affect indexes and constraints.** Most engines index NULLs (Oracle does not unless the index is composite), UNIQUE allows multiple NULLs in PostgreSQL, MySQL and SQLite, and only one in SQL Server.

BEST PRACTICE · Decide nullability at design time
Every column is NOT NULL until you can name, in one sentence, why a row may legitimately have no value. “It might be missing” is a data-quality problem, not a schema feature — and a NOT NULL constraint is the cheapest data-quality job you will ever ship.

1.7 A type does not just cost space — it costs space everywhere

A column's type shows up in the table, in every index that contains it, in every join hash table and sort buffer that touches it, and in every backup and replica stream. Estimate before you declare:

arithmetic worth doing before the table exists

```
rows            = 100,000,000
column          = one BIGINT key   -> 8 bytes/row  = 0.8 GB
same column     = INT key          -> 4 bytes/row  = 0.4 GB
one secondary index on that key    -> +4 to +8 bytes/row + page overhead ≈ 0.9 GB
30-day backup retention, 2 replicas -> multiply that difference again

-- rule of thumb: total ≈ rows × (row width + ~10%) × (1 + number of indexes)
```

TIP · The 4-byte question
For every integer key ask: will this table ever exceed 4.29 billion rows (2.1 billion signed)? If not, INT. Analytics tables often never do; append-only event tables usually will. Changing a key type later means rebuilding the table and every index that references it.

1.8 Choosing a type: the decision order

1. **Name the meaning.** Money? Instant in time? Local date? Category? Identifier? Free text? Measurement? The meaning picks the family, not the other way round.
2. **Pick exactness before size.** Money and counters are exact: DECIMAL or INTEGER, never FLOAT.
3. **Estimate the range.** Signed TINYINT ±127 may be perfect for a 1–5 rating and wrong for a size in bytes.
4. **Decide the time semantics.** An instant (TIMESTAMPTZ / DATETIMEOFFSET) or a wall-clock value (DATE / DATETIME)? See Part 4 — this decision is the hardest to reverse.
5. **Decide text length policy.** A code with a fixed vocabulary is CHAR(n), a bounded label is VARCHAR(n), unbounded prose is TEXT and belongs in a side table if it is large.
6. **Constrain the domain.** CHECK, ENUM or a lookup table. If the values can change without a release, use a lookup table; if they change with a release, ENUM is acceptable.
7. **Only then optimise for bytes.** Size tuning comes last and matters least — correctness first, portability second, bytes third.

1.9 Reading a real schema

Here is a small warehouse-style schema. Notice how many decisions are visible in the declarations alone — and how a reviewer can question each one.

postgresql 16 · a schema where every type is a decision

```
CREATE TABLE dim_customer (
    customer_sk    INTEGER      PRIMARY KEY,              -- surrogate key, warehouse-local
    customer_id    VARCHAR(20)  NOT NULL UNIQUE,          -- natural key, fixed-width code
    full_name      VARCHAR(120) NOT NULL,
    email          VARCHAR(254) NOT NULL,                 -- 254 = RFC max practical length
    country_code   CHAR(2)      NOT NULL,                 -- ISO 3166-1 alpha-2
    signup_date    DATE         NOT NULL,
    is_active      BOOLEAN      NOT NULL DEFAULT TRUE,
    loyalty_tier   VARCHAR(10)  NOT NULL DEFAULT 'none'   -- low-cardinality label
);

CREATE TABLE fact_sales (
    sale_id        BIGINT       PRIMARY KEY,
    sold_at        TIMESTAMPTZ  NOT NULL,                 -- an instant, not a wall clock
    business_date  DATE         NOT NULL,                 -- pre-computed reporting grain
    customer_sk    INTEGER      NOT NULL REFERENCES dim_customer(customer_sk),
    product_sk     INTEGER      NOT NULL,
```

```
    quantity       INTEGER      NOT NULL CHECK (quantity > 0),
    unit_price     DECIMAL(12,2) NOT NULL,                -- exact money
    discount_pct   DECIMAL(5,2)  NOT NULL DEFAULT 0,
    net_amount     DECIMAL(14,2) NOT NULL,                -- pre-computed to avoid re-deriving
    currency       CHAR(3)      NOT NULL DEFAULT 'INR'
);
```

Why surrogate INTEGER keys?
Filtering and joining on a 4-byte integer beats a 20-byte natural key, and dimension keys never overflow in practice. The natural key stays, with a UNIQUE index, for traceability.

Why both sold_at and business_date?
sold_at answers “when exactly”; business_date answers “which day does finance count this in”. Deriving the second from the first inside every query is slower and drifts.

Why DECIMAL for discount_pct?
Percentages look harmless as floats, but repeated multiplication accumulates error. Fixed scale keeps totals reproducible — a hard requirement for reconciliation.

Why VARCHAR(254) for email?
The limit documents the domain and blocks 10 MB junk. PostgreSQL will not perform better, but the next engineer learns the intent — that is the point.

WARNING · The one thing you cannot fix later
Choosing DATETIME where you needed an instant, or FLOAT where you needed money, is not a typing mistake — it is a data-loss event. Every value written before the fix is already wrong. Fix those two choices at design time, always.

1.10 Part 1 in one page

If you remember one thing	Do this
A type is a contract with four parts	Check encoding, domain, operations and ordering before signing it
Analytics fails quietly	Prefer exact types and explicit casts in anything that produces a number
Families are portable, names are not	Design in families, translate per engine (Part 8)
Fixed, variable, off-page	Keep large text and blobs out of the hot row; never SELECT *
NULL is a value, not a zero	Decide nullability, use IS NULL, use COALESCE for display
Cost is everywhere	Estimate rows × width × indexes before declaring BIGINT

Table 1.4 — Part 1 summary.

NOTES · PART 1 · FOUNDATIONS

PART 2

Numeric Types

Integers, exact decimals and floating point — and the three bugs they cause

Numbers are the easiest types to get nearly right and the easiest to make quietly wrong. This part covers every numeric family, the exact ranges and byte costs, and the four failures that show up in real analysis: overflow, integer division, float error and scale mismatch.

2.1 The numeric family tree

Branch	Behaviour	Types	Use for
Integer	Whole numbers, exact, no fractional part	TINYINT / SMALLINT / INT / BIGINT, SERIAL, IDENTITY	IDs, counts, quantities, flags
Exact decimal	Fixed number of decimal digits, exact arithmetic	DECIMAL(p,s), NUMERIC(p,s), NUMBER(p,s), MONEY	Money, tax, percentages, measurements
Approximate	Binary floating point, fast, not exact	FLOAT, REAL, DOUBLE PRECISION, BINARY_FLOAT / BINARY_DOUBLE	Scientific data, ratios, ML features
Bit	Bit strings, used as flags or masks	BIT, BIT(n), VARBIT	Feature flags, permission masks
Auto-numbering	Sequence-backed generators	AUTO_INCREMENT, IDENTITY, SERIAL, SEQUENCE	Surrogate keys

Table 2.1 — Five branches of the numeric tree.

2.2 Integer types: exact by definition

Integers are stored in two's complement. A signed type of n bits holds $-2^{(n-1)}$ to $2^{(n-1)}-1$; unsigned holds 0 to $2^{n}-1$. Know the four boundaries below and you will never pick a type too small.

Type	Bits	Signed range	Unsigned range	Bytes
TINYINT	8	−128 ... 127	0 ... 255	1
SMALLINT	16	−32,768 ... 32,767	0 ... 65,535	2
MEDIUMINT	24	−8,388,608 ... 8,388,607	0 ... 16,777,215	3
INT / INTEGER	32	−2,147,483,648 ... 2,147,483,647	0 ... 4,294,967,295	4
BIGINT	64	$\pm 9.22 \times 10^{18}$	0 ... 1.84×10^{19}	8

Table 2.2 — Integer widths. MEDIUMINT is MySQL-only.

- **PostgreSQL** has SMALLINT, INTEGER, BIGINT; its SMALLSERIAL, SERIAL and BIGSERIAL are shorthand for 2, 4 and 8-byte columns with an owned sequence.
- **SQL Server** has TINYINT (0–255, unsigned only), SMALLINT, INT and BIGINT. There is no UNSIGNED keyword.
- **Oracle** has no integer types at all: NUMBER(38) is the closest, and NUMBER(p,0) is how you declare a p-digit integer. Storage is variable-length, 1–22 bytes.
- **SQLite** has one INTEGER type with up to 8 bytes; INTEGER PRIMARY KEY becomes the rowid.
- **MySQL** adds UNSIGNED and ZEROFILL modifiers, plus MEDIUMINT.

Picking an integer width

Column	Right type	Why not bigger
order_id on a retailer doing 5,000 orders/day	INT	~1.2 M rows/year; INT lasts centuries
event_id on a clickstream	BIGINT	Billions per month; wrap-around risk is real
rating 1–5	TINYINT	1 byte, CHECK (rating BETWEEN 1 AND 5)
age in years	SMALLINT	TINYINT is fine but SMALLINT leaves room for 200+ records
year	SMALLINT — or DATE	A year is not an integer conceptually; see Part 4
quantity	INT	Bulk orders exceed 32,767 regularly
flags (is_active)	BOOLEAN, not INTEGER	Meaning, not bytes, is the reason

Table 2.3 — Width is a range decision, not a style decision.

COMMON TRAP · Unsigned is not portable and not free
UNSIGNED doubles the positive range but removes negative numbers, breaks subtraction (0 − 1 raises an error or wraps), and has no equivalent in PostgreSQL, SQL Server or Oracle. Use it for values that cannot ever be negative and where you control every consumer: status flags, counts, quantities.

Overflow: the bug that only appears in year three

`universal · overflow behaves differently per engine`

```
-- MySQL, non-strict mode: silently clamps to the maximum
CREATE TABLE t (stock SMALLINT);           -- max 32,767
INSERT INTO t VALUES (40000);              -- stored as 32767, warning raised
SELECT * FROM t;                            -- 32767 ← data loss, no error

-- PostgreSQL: loud error instead, which is what you want
INSERT INTO t VALUES (40000);
ERROR:  smallint out of range

-- INTEGER wrap-around on a counter (MySQL, non-strict)
UPDATE counters SET hits = hits + 1;        -- 2147483647 + 1 → -2147483648
-- Fix: BIGINT for any monotonic counter that runs forever.
```

BEST PRACTICE · The counter rule
Any column that only ever increases — event IDs, page views, invoice numbers, sequence values — is BIGINT. A 4-byte INT reaching 2.1 billion is not an edge case on a table growing 5 million rows a day (420 days).

2.3 Auto-numbering: AUTO_INCREMENT, IDENTITY, SERIAL

Surrogate keys need a generator. Every engine has one; the semantics differ more than the syntax suggests, especially around gaps, resets and explicit inserts.

Engine	Declare	Notes
MySQL	id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY	Counter lives in table metadata; gaps after rollbacks; innodb_autoinc_lock_mode controls concurrency
PostgreSQL	id BIGINT GENERATED ALWAYS AS IDENTITY	Preferred since PG 10. GENERATED BY DEFAULT allows explicit inserts; SERIAL is the legacy equivalent
SQL Server	id BIGINT IDENTITY(1,1) PRIMARY KEY	Gaps are normal and expected; SET IDENTITY_INSERT ON to restore data
Oracle 12c+	id NUMBER GENERATED ALWAYS AS IDENTITY	Legacy code uses sequences with triggers; IDENTITY is now standard
SQLite	id INTEGER PRIMARY KEY	Alias for rowid; AUTOINCREMENT adds a separate monotonic counter (rarely needed)

Table 2.4 — Six ways to say “number my rows”.

sequences are objects, not column properties

```
-- portable-ish sequence usage (PostgreSQL / Oracle / SQL Server 2012+)
CREATE SEQUENCE order_seq START WITH 1000 INCREMENT BY 1;

INSERT INTO orders (order_id, customer_id)
VALUES (NEXT VALUE FOR order_seq, 42);        -- SQL Server / Oracle syntax

-- PostgreSQL equivalent
INSERT INTO orders (order_id, customer_id)
VALUES (nextval('order_seq'), 42);
```

WARNING · Identity columns are not gapless
A rolled-back transaction consumes its number. Auditors who require gapless invoice numbers need a different mechanism (a locked counter table or a sequence with a business rule), not a bigger identity column. Never promise sequential meaning to a number that is only unique.

2.4 DECIMAL and NUMERIC: exact by declaration

DECIMAL(p,s) stores **p** total digits, of which **s** are to the right of the decimal point. That leaves p−s integer digits. The scale is enforced on write: the engine rounds values with more decimals and rejects values with too many integer digits.

Declaration	Integer digits	Decimals	Largest value	Use for
DECIMAL(5,2)	3	2	999.99	Percentages, small rates
DECIMAL(7,2)	5	2	99,999.99	Unit prices
DECIMAL(10,2)	8	2	99,999,999.99	Order totals
DECIMAL(12,2)	10	2	9,999,999,999.99	Daily revenue
DECIMAL(19,4)	15	4	999,999,999,999.9999	Banking amounts, FX rates
DECIMAL(38,10)	28	10	1e28 with 10 dp	Scientific/financial analytics
DECIMAL(38,0)	38	0	1e38	Big counters, hashes as numbers

Table 2.5 — Choose p and s from the business rule, not by habit.

universal · scale is enforced, and it rounds

```
-- What the scale actually does on write
CREATE TABLE price_demo (price DECIMAL(5,2));

INSERT INTO price_demo VALUES (123.456);   -- → 123.46 (rounded, not truncated)
INSERT INTO price_demo VALUES (1234.00);   -- → error on PostgreSQL: numeric field overflow
-- MySQL non-strict: 999.99 (clamped!)
INSERT INTO price_demo VALUES (99);        -- → 99.00
```

```
-- Scale is part of the type: DECIMAL(5,2) ≠ DECIMAL(5,3)
SELECT CAST(1 AS DECIMAL(5,2)) / CAST(3 AS DECIMAL(5,2));  -- 0.33, not 0.3333
```

- **Limits:** MySQL allows p ≤ 65, s ≤ 30. PostgreSQL NUMERIC allows up to 131,072 digits before and 16,383 after the point (declared precision is only enforced when you specify it). SQL Server allows precision 1–38. Oracle NUMBER allows 1–38 significant digits. SQLite stores NUMERIC as REAL or INTEGER internally, so exactness is best-effort unless you use STRICT tables with careful values.
- **Storage:** MySQL packs DECIMAL in 4-byte groups (about 4 bytes per 9 digits); each decimal digit costs roughly 0.44 bytes. SQL Server uses 5, 9, 13 or 17 bytes by precision. PostgreSQL NUMERIC is variable, close to 2 bytes per 4 digits plus overhead. Oracle NUMBER is 1–22 bytes.
- **Division inherits scale.** In MySQL, DECIMAL division adds 4 digits of scale (div_precision_increment) unless you specify otherwise. Always CAST the result to the scale the report needs.

When the scale is wrong, the fix is expensive

universal · precision loss is not reversible

```
-- Wrong: two decimals for a unit price recorded per thousand units
unit_price DECIMAL(10,2)          -- 0.0125 becomes 0.01, a 20% error at volume

-- Right: carry the precision you actually receive
unit_price DECIMAL(12,6)

-- Wrong: FX conversions rounded at every hop
amount_inr = ROUND(amount_usd * rate, 2)                  -- drift accumulates per row
-- Right: keep full precision internally, round once at presentation
ROUND(SUM(amount_usd * rate), 2)
```

BEST PRACTICE · Money rule of thumb
Money is DECIMAL(19,4) or DECIMAL(12,2) depending on currency magnitude, always paired with an ISO-4217 CHAR(3) currency column. Never store two currencies in one amount column without the currency, and never store money as a float, no matter what the vendor's sample schema does.

2.5 FLOAT, REAL and DOUBLE PRECISION

Approximate types follow IEEE 754: a sign, an exponent and a mantissa. They cover an enormous range in a few bytes, but they cannot represent most decimal fractions exactly, because binary fractions are sums of 1/2, 1/4, 1/8... and 0.1 is not.

Type	Bytes	Significant digits	Approx. range	Aliases
REAL / FLOAT4	4	~7	1e−38 to 1e38	MySQL FLOAT, Oracle BINARY_FLOAT, SQL Server REAL
DOUBLE PRECISION	8	~15–17	1e−308 to 1e308	MySQL DOUBLE, Oracle BINARY_DOUBLE, SQL Server FLOAT
FLOAT(n)	4 or 8	n ≤ 24 → 4 bytes	as above	SQL Server: precision-driven; Oracle FLOAT is binary precision

Table 2.6 — Approximate types and their aliases.

universal · the classic float demonstrations

```
SELECT 0.1 + 0.2 = 0.3;             -- FALSE on every engine
SELECT 0.1::float8 + 0.2::float8;    -- 0.30000000000000004 (PostgreSQL output)

-- Accumulation: 10 million additions of 0.01
SELECT SUM(x) FROM (SELECT 0.01 AS x FROM generate_series(1,10000000)) t;
-- 99999.99999999975 instead of 100000.00
```

```
-- Comparisons must use a tolerance, never equality
SELECT * FROM measurements
WHERE ABS(actual - expected) < 1e-9;

-- Money as float: rounding drift that a finance team will find
SELECT ROUND(SUM(amount)::numeric, 2) FROM payments;   -- 1,048,576.00 vs 1,048,575.97
```

GO DEEPER · Why 0.1 is not 0.1
$0.1_{10} = 0.0001100110011..._2$, a repeating fraction in base two. An 8-byte double keeps 53 bits and rounds the rest, so the stored value is 0.1000000000000000055511151231257827021181583404541015625. Every arithmetic operation then carries that tiny error, and the error is invisible until you compare, group or sum it.

- **Use floats for** physical measurements, sensor readings, probabilities, ML features, ratios where 6–15 significant digits are enough, and any value that is already an approximation.
- **Never use floats for** money, invoice totals, tax, points, quotas, or anything reconciled to the cent. If a human will check the arithmetic down to paise, the type is DECIMAL.
- **Never join or deduplicate on a float.** Two values that print the same can differ in the last bit. Round to a fixed scale first, or use a tolerance.
- **Watch the float-to-integer conversion.** CAST(2.9999999 AS INT) truncates to 2, not 3 (PostgreSQL rounds: 3). Use ROUND explicitly.

2.6 MONEY types (and why most teams avoid them)

Engine	Type	Bytes / range	Verdict
SQL Server	MONEY / SMALLMONEY	8 / 4 bytes, 4 decimal places, max ~922 trillion	Faster but fixed at 4 dp and locale-independent; prefer DECIMAL(19,4)
PostgreSQL	MONEY	8 bytes, locale-aware formatting, fractional cents	Deprecated in practice: rounds to locale, does not play well with numeric functions
MySQL	None	—	Use DECIMAL(19,4)
Oracle	None	—	Use NUMBER(19,4)
SQLite	None	—	Store integer minor units (paise/cents) for exactness

Table 2.7 — MONEY exists, and usually you should not use it.

TIP · The integer-cents pattern
Store amounts as BIGINT minor units (12.34 → 1234) when you need exact, engine-portable arithmetic: no scale ambiguity, no rounding mode disputes, exact equality. Divide by 100 only in the presentation layer — and only once.

2.7 Integer arithmetic and division: the silent zero

Integer division truncates. In SQL, the result type of a/b follows the operands, not your expectation, which is why a percentage column quietly becomes 0 or 1.

universal · division depends on operand types, not on hope

```
SELECT 7 / 2;                    -- MySQL: 3 SQL Server: 3 PostgreSQL: 3
SELECT 7 / 2.0;                  -- 3.5 (one operand decimal → decimal result)
SELECT 7.0 / 2;                  -- 3.5
SELECT CAST(7 AS DECIMAL(10,2)) / 2;   -- 3.500000 (scale per engine rules)

-- The classic broken conversion rate
SELECT clicks / impressions AS ctr FROM campaign;          -- always 0
SELECT clicks * 100.0 / NULLIF(impressions, 0) AS ctr_pct  -- correct, and safe
FROM campaign;

-- Integer averages lose the fraction
```

```
SELECT AVG(rating) FROM reviews;      -- PostgreSQL: 4.25 (numeric)
-- SQL Server: 4 (integer AVG!) → CAST first
SELECT AVG(CAST(rating AS DECIMAL(5,2))) FROM reviews;
```

COMMON TRAP · SQL Server integer AVG
In SQL Server, AVG over an INT column returns an INT, so 4.25 becomes 4. MySQL, PostgreSQL and Oracle return a decimal. If your dashboards must agree across engines, divide by 1.0 or cast the column explicitly.

2.8 Rounding, truncation and the functions you need

Intent	MySQL	PostgreSQL	SQL Server	Oracle
Round half away from zero	ROUND(x, n)	ROUND(x::numeric, n)	ROUND(x, n)	ROUND(x, n)
Truncate toward zero	TRUNCATE(x, n)	TRUNC(x, n)	ROUND(x, n, 1)	TRUNC(x, n)
Always up / down	CEIL / FLOOR	CEIL / FLOOR	CEILING / FLOOR	CEIL / FLOOR
Modulo	MOD(a,b) or a % b	MOD(a,b) or a % b	a % b	MOD(a,b)
Power / square root	POWER, SQRT	POWER, SQRT	POWER, SQRT	POWER, SQRT
Absolute value	ABS(x)	ABS(x)	ABS(x)	ABS(x)
Safe divide	NULLIF(b,0)	NULLIF(b,0)	NULLIF(b,0)	NULLIF(b,0)
Cast to integer	CAST(x AS SIGNED)	x::integer or CAST	CAST(x AS INT)	CAST(x AS NUMBER)

Table 2.8 — Numeric functions you will actually use.

Rounding modes matter legally and financially. MySQL and PostgreSQL ROUND use half away from zero for exact types; Python and many BI tools use banker's rounding (half to even). 2.5 becomes 3 in SQL and 2 in pandas. When a finance team says “the numbers don't match the spreadsheet”, this is usually why.

2.9 BIT and bit masks

mysql & postgresql · bit strings

```
-- MySQL: BIT(M), M ≤ 64, stored as a bit string
CREATE TABLE feature_flags (user_id INT, flags BIT(8));
INSERT INTO feature_flags VALUES (1, b'00000101');       -- flags 0 and 2 set
SELECT user_id FROM feature_flags WHERE flags & b'00000100';  -- bit 2 present

-- PostgreSQL: the native type is BIT / VARBIT, tested with B'...'
SELECT B'1010' & B'0010';         -- 0010

-- Modern alternative: separate BOOLEAN columns or a JSON array of feature names
-- Bit masks are compact but opaque to BI tools and unreadable in a WHERE clause
```

NOTE · BIT in SQL Server means something else
In SQL Server, BIT is a 1-bit boolean-like type (0, 1 or NULL). In MySQL and PostgreSQL, BIT(n) is a bit string. Same word, different concept — a classic portability trap.

2.10 Engine cheat sheet: numeric types

Need	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
1-byte integer	TINYINT	—	TINYINT (0–255)	NUMBER(3)	INTEGER
2-byte integer	SMALLINT	SMALLINT	SMALLINT	NUMBER(5)	INTEGER
4-byte integer	INT	INTEGER	INT	NUMBER(10)	INTEGER
8-byte integer	BIGINT	BIGINT	BIGINT	NUMBER(19)	INTEGER
Exact decimal	DECIMAL(p,s)	NUMERIC(p,s)	DECIMAL(p,s)	NUMBER(p,s)	NUMERIC
4-byte float	FLOAT	REAL	REAL	BINARY_FLOAT	REAL
8-byte float	DOUBLE	DOUBLE PRECISION	FLOAT	BINARY_DOUBLE	REAL
Money	—	MONEY (avoid)	MONEY	—	—
Auto key	AUTO_INCREMENT	IDENTITY / SERIAL	IDENTITY	IDENTITY / SEQUENCE	INTEGER PRIMARY KEY

Table 2.9 — The same idea, six spellings.

2.11 Numeric pitfalls checklist

Symptom	Cause	Fix
Total off by a cent after many rows	FLOAT used for money	DECIMAL(19,4) or integer minor units
Percentage always 0	Integer division	Multiply by 100.0 or CAST to DECIMAL
“Value too long” for a price	Scale p−s too small	Widen the integer part: DECIMAL(12,2)
Negative stock after a sale	UNSIGNED subtraction	SIGNED type plus a CHECK (qty ≥ 0)
Counter became negative	INT overflow	BIGINT
Duplicates after a join	Float equality between engines	Round to a fixed scale before joining
Report and spreadsheet disagree	Different rounding modes	ROUND in SQL at the agreed scale, once
Averages too low	NULL rows skipped by AVG	COALESCE the value, or document the denominator

Table 2.10 — Eight symptoms and their numeric causes.

TIP · Try it

Create a table with DECIMAL(5,2) and insert 999.995, 1234.00 and -0.001. Note what each engine stores. Then repeat with FLOAT and sum 10,000 rows of 0.01 to see the drift for yourself.

NOTES · PART 2 · NUMERIC

PART 3

Character & String Types

Fixed, variable, unlimited — plus charsets, collations and search

Text looks like the simplest type and is the source of the most surprising behaviour: limits counted in bytes instead of characters, trailing spaces that break equality, case-insensitive collations, emoji that need four bytes, and indexes that silently do not apply. This part makes all of it predictable.

3.1 The three text shapes

Shape	Declaration	Storage	Semantics	Use for
Fixed	CHAR(n) / NCHAR(n)	Exactly n characters, space-padded	Pad-and-compare: 'A' = 'A '	True fixed-width codes: ISO country CHAR(2), currency CHAR(3)
Variable, bounded	VARCHAR(n) / NVARCHAR(n)	Length prefix + actual characters	Trailing spaces kept as written (MySQL trims on write for CHAR only)	Codes, names, emails, labels — anything with a known ceiling
Variable, large	TEXT / LONGTEXT / VARCHAR(MAX) / CLOB	Pointer in row, payload off-page	Same comparison rules, different I/O cost	Descriptions, comments, JSON snapshots, log payloads

Table 3.1 — Three shapes cover every text column you will design.

3.2 CHAR versus VARCHAR: the padding problem

CHAR(n) pads shorter values with spaces. Standard SQL says the padding is not significant for comparison, which means 'AB' = 'AB ' is TRUE. Engines implement this differently, and the difference bites when you move data or compare across joins.

universal · same declaration, three behaviours

```
-- MySQL: CHAR pads and strips trailing spaces on retrieval
CREATE TABLE t (code CHAR(4));
INSERT INTO t VALUES ('AB');
SELECT CONCAT('[', code, ']') FROM t;   -- [AB] (padding stripped on read)
SELECT LENGTH(code) FROM t;              -- 2

-- PostgreSQL: CHAR pads and keeps padding; comparisons ignore it
SELECT 'AB'::char(4) = 'AB';             -- true
SELECT length('AB'::char(4));            -- 2 (bpchar strips trailing spaces in length)
SELECT octet_length('AB'::char(4));      -- 4

-- SQL Server: CHAR pads, and trailing spaces are ignored in comparisons
-- Oracle: CHAR pads to the declared length on storage and on read
SELECT DUMP(code) FROM t;                -- shows 'AB ' with two spaces
```

BEST PRACTICE · Use VARCHAR(n) unless the value is genuinely fixed-width

CHAR(2) for ISO country codes is correct and self-documenting. CHAR(50) for a name is a bug: it wastes space, makes every row the same width, and produces trailing-space comparisons that break equality in application code.

3.3 Bytes, characters and the utf8mb4 rule

VARCHAR(n) counts characters in standard SQL, but MySQL's limit is expressed in bytes, so a VARCHAR(255) in utf8mb4 can occupy up to 1,020 bytes. This is why some ALTERs and index creations fail with “specified key was too long”.

Charset	Bytes per character	Can store	Cannot store
latin1	1	Western European text	Devanagari, CJK, emoji, curly quotes
utf8mb3 (MySQL “utf8”)	1–3	Most BMP characters	Emoji, supplementary planes
utf8mb4	1–4	Everything, including emoji and Indic scripts	—
UTF-16 (NVARCHAR in SQL Server)	2–4	Full Unicode	—
AL32UTF8 (Oracle)	1–4	Full Unicode	—

Table 3.2 — Choose utf8mb4 and never think about it again.

- **MySQL:** set the database, table and connection to utf8mb4. The old “utf8” alias is utf8mb3 and cannot store emoji — the classic “Incorrect string value” error.
- **SQL Server:** VARCHAR is single-byte (code page dependent) and NVARCHAR is Unicode. Any multilingual or emoji-bearing column must be NVARCHAR — there is no way to add Unicode later without a rewrite.
- **Oracle:** VARCHAR2 length semantics are BYTE by default; use VARCHAR2(50 CHAR) or set NLS_LENGTH_SEMANTICS=CHAR.
- **SQLite & PostgreSQL:** UTF-8 always. PostgreSQL databases are created with an encoding that cannot be changed without a dump and reload, so create with UTF8 from the start.

engine-specific · charset declarations that prevent mojibake

```sql
-- MySQL: mixing charsets is the classic mojibake source
SHOW VARIABLES LIKE 'character_set%';
CREATE TABLE messages (
    body VARCHAR(280) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
) DEFAULT CHARSET = utf8mb4;

-- SQL Server: the N prefix is required for Unicode literals
SELECT * FROM messages WHERE body = N'□□□□□□';
-- ⚠ without the N prefix, the literal is converted to the database code page

-- Oracle: count characters, not bytes
CREATE TABLE t (name VARCHAR2(50 CHAR));
```

WARNING · Changing a charset is a table rebuild
Converting latin1 to utf8mb4 rewrites every row and rebuilds every index. Do it while you can, on a new schema where possible. On a large production table it is a maintenance-window job, which is exactly why the choice must be right on day one.

3.4 Collation: how text sorts and compares

Collation decides case sensitivity, accent sensitivity and sort order. It is part of the column's behaviour, not just its storage, so two columns of the same type can compare differently.

Collation (MySQL)	Case	Accents	Typical behaviour
utf8mb4_0900_ai_ci	Insensitive	Insensitive	Default in MySQL 8; 'Mumbai' = 'mumbai'
utf8mb4_0900_as_cs	Sensitive	Sensitive	Exact byte-ish comparison
utf8mb4_bin	Sensitive	Sensitive	Binary comparison, fastest but locale-blind
utf8mb4_0900_ai_ci + unicode_ci	Insensitive	Insensitive	Language-aware sorting

Table 3.3 — MySQL 8 collations you will meet. PostgreSQL and SQL Server use similar naming schemes with different defaults.

universal · case-insensitive uniqueness, five ways

```
-- Case sensitivity is a collation decision, not a coding style
CREATE TABLE users (email VARCHAR(254) COLLATE utf8mb4_0900_ai_ci UNIQUE); -- dedupes Foo@x and foo@x
CREATE TABLE codes (code VARCHAR(10) COLLATE utf8mb4_bin UNIQUE);          -- keeps case distinct

-- Cross-engine equivalents
-- PostgreSQL: CREATE COLLATION or citext extension
CREATE EXTENSION IF NOT EXISTS citext;
CREATE TABLE users (email CITEXT UNIQUE);            -- case-insensitive comparisons

-- SQL Server: choose at database/column level
email NVARCHAR(254) COLLATE Latin1_General_CI_AS    -- CI = case-insensitive, AS = accent-sensitive

-- Oracle: NLS_COMP / NLS_SORT, or a function-based unique index
CREATE UNIQUE INDEX ux_email ON users (LOWER(email));
```

COMMON TRAP · Mixing collations breaks indexes and joins
Comparing a Latin1_General_CI_AS column with a Latin1_General_CS_AS column in SQL Server raises “cannot resolve the collation conflict”. In MySQL, a join between a utf8mb4 column and a utf8mb3 column forces a conversion and can disable the index. Keep one collation per database.

3.5 Length semantics: LENGTH, CHAR_LENGTH and the 65,535 limit

universal · limits and length functions

```
SELECT LENGTH('□□□□□□');        -- MySQL: 18 bytes PostgreSQL: 18 bytes
SELECT CHAR_LENGTH('□□□□□□');  -- 6 characters (MySQL, PostgreSQL)
-- SQL Server: LEN counts characters but ignores trailing spaces; DATALENGTH counts bytes
-- Oracle: LENGTH counts characters, LENGTHB counts bytes

-- MySQL limits by type (bytes, before charset multiplication)
VARCHAR(n)      n ≤ 65,535 per row, row limit 65,535 bytes overall
TINYTEXT        255 bytes
TEXT            65,535 bytes
MEDIUMTEXT      16,777,215 bytes
LONGTEXT        4,294,967,295 bytes

-- SQL Server
VARCHAR(n)      n ≤ 8,000          NVARCHAR(n) n ≤ 4,000
VARCHAR(MAX)    2 GB               NVARCHAR(MAX) 2 GB

-- PostgreSQL
VARCHAR(n) / TEXT   effectively unlimited (1 GB per field)
CHAR(n)             n ≤ 10,485,760, blank-padded
```

GO DEEPER · Why PostgreSQL VARCHAR(n) does not save space
In PostgreSQL, VARCHAR(n), VARCHAR and TEXT have identical storage and identical performance: the length is a constraint, not an allocation. Use VARCHAR(n) when the limit documents the domain; never use it hoping for speed. In MySQL the limit changes the maximum row size, so there it does affect layout.

3.6 Text search: LIKE, regex and full-text

Pattern	Sargable?	Notes
col = 'value'	Yes	Index seek if the collation matches
col LIKE 'abc%'	Yes	Left-anchored prefix can use a B-tree index
col LIKE '%abc%'	No	Full scan on every engine; use a trigram/full-text index
LOWER(col) = 'abc'	No	Unless you create an expression index on LOWER(col)
col ~ 'regex'	No	Regex is a scan; PostgreSQL can use pg_trgm for some patterns
FULLTEXT / tsvector	Specialised	Inverted index; language-aware stemming

Table 3.4 — What keeps an index usable.

engine-specific · making text search fast

```
-- Expression index keeps case-insensitive search sargable (PostgreSQL)
CREATE INDEX ix_users_email_lower ON users (LOWER(email));
SELECT * FROM users WHERE LOWER(email) = 'foo@x.com';      -- uses the index

-- MySQL 8 functional index
ALTER TABLE users ADD INDEX ix_email_lower ((LOWER(email)));

-- Full-text search
-- MySQL
CREATE FULLTEXT INDEX ft_body ON articles (body);
SELECT * FROM articles WHERE MATCH(body) AGAINST ('data type' IN NATURAL LANGUAGE MODE);

-- PostgreSQL
CREATE INDEX ix_body_fts ON articles USING GIN (to_tsvector('english', body));
SELECT * FROM articles WHERE to_tsvector('english', body) @@ plainto_tsquery('english','data type');
```

3.7 Practical rules for text columns

Rule	Reason
Use VARCHAR(n) with a limit that means something	254 for emails, 2/3 for ISO codes, 120 for names — the limit is documentation
Never use CHAR(n) for names or free text	Padding, wasted bytes, trailing-space comparisons
Keep TEXT/CLOB out of the hot table	Off-page reads; move to a detail table if rarely selected
Always specify the charset explicitly on MySQL	The server default silently changes with upgrades and dumps
Prefer unicode everywhere, once	NVARCHAR/utf8mb4 from day one; conversions are rebuilds
Store one value per column	'Delhi, Mumbai' in a city column removes every filter and group-by
Normalise obvious lookups	Status and country values belong in a lookup table, not free text
Do not put numbers and dates in VARCHAR	Sorting becomes lexical: '10' < '9', 'Apr' < 'Mar'
Trim on write, not on read	TRIM() inside a WHERE clause disables the index

Table 3.5 — Nine rules that prevent most text-type defects.

3.8 Engine cheat sheet: character types

Need	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
Fixed length	CHAR(n)	CHAR(n)	CHAR(n) / NCHAR(n)	CHAR(n)	TEXT
Bounded variable	VARCHAR(n)	VARCHAR(n)	VARCHAR(n) / NVARCHAR(n)	VARCHAR2(n)	TEXT
Unbounded text	TEXT / LONGTEXT	TEXT	VARCHAR(MAX) / NVARCHAR(MAX)	CLOB / NCLOB	TEXT
Unicode	utf8mb4 collation	UTF-8 database	NVARCHAR / NCHAR	NVARCHAR2 / AL32UTF8	always UTF-8
Binary text	BLOB / VARBINARY	BYTEA	VARBINARY(MAX)	BLOB / RAW	BLOB
Case-insensitive	collation ..._ci	citext extension	collation ..._CI	NLS_SORT setting	COLLATE NOCASE

Table 3.6 — Character types across engines.

TIP · Try it

Insert an emoji into a utf8mb3 and a utf8mb4 column in MySQL and compare the errors. Then insert 'A' into CHAR(4) and VARCHAR(4) on PostgreSQL and compare octet_length(). The difference is the whole lesson of this part.

NOTES · PART 3 · CHARACTER

PART 4

Date & Time Types

Instants, wall clocks, durations — the type decision hardest to reverse

Dates and times cause more production incidents than every other type family combined. The reason is simple: the same instant has many correct spellings. This part separates the two questions every temporal column must answer — **what is it** (an instant, a local date, a duration) and **whose clock** (UTC, server, user) — and then shows the exact types, ranges and functions per engine.

4.1 The four temporal concepts

Instant (point in time) A moment on the world timeline, independent of any clock. Stored as UTC plus an offset, or as a normalised UTC value. Type names contain **TZ**, **OFFSET** or **WITH TIME ZONE**.	**Wall clock (local)** What a calendar on a wall shows: 2026-10-27 09:30 in Mumbai. No offset attached, so it is not a unique instant. Type names are DATE, DATETIME, TIMESTAMP without time zone.
Calendar date A day with no time at all: birth date, invoice date, business date. Use DATE, never DATETIME at midnight — midnight does not always exist in every time zone (DST).	**Duration** A length of time, not a moment: 90 days, 3 hours 15 minutes. Use INTERVAL, or an integer count of a declared unit.

4.2 The temporal type catalogue

Type	What it is	Range / precision	Use for
DATE	Calendar date, no time	4713 BC – 5874897 AD (PG); 1000–9999 (MySQL DATE)	Business dates, birthdays, invoice dates
TIME	Time of day, no date	00:00:00 – 23:59:59.999999	Opening hours, schedule slots
DATETIME	Date + wall-clock time, no zone	1000-01-01 – 9999-12-31 (MySQL); 1753+ (SQL Server)	Legacy data, purely local schedules
TIMESTAMP	Date + time; zone-aware in PostgreSQL (stored UTC), naive in MySQL and SQL Server	1970–2038 (MySQL TIMESTAMP); 4713 BC+ (PG)	Event times — but check the engine semantics
TIMESTAMPTZ	Instant: stored UTC, rendered in the session zone	4713 BC – 294276 AD	Anything that happens once, globally
DATETIMEOFFSET	Instant plus the originating offset	0001-01-01 – 9999-12-31	Cross-region events in SQL Server
INTERVAL	Duration	±178,000,000 years in PG	Ages, windows, SLA durations
YEAR	Four-digit year in 1 byte (MySQL)	1901–2155	Legacy dimension tables

Table 4.1 — Temporal types. The same word means different things per engine.

WARNING · The single most dangerous word in SQL: TIMESTAMP

PostgreSQL's TIMESTAMP is naive, TIMESTAMPTZ is an instant. MySQL's TIMESTAMP is an instant (converted to UTC, rendered in the session zone) and DATETIME is naive. SQL Server's TIMESTAMP is not a date at all — it is a rowversion binary. Oracle's TIMESTAMP has no zone; TIMESTAMP WITH TIME ZONE does. If a schema review finds an unqualified TIMESTAMP, that is the first question to ask.

4.3 Storage cost per engine

Type	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
DATE	3 bytes	4 bytes	3 bytes	7 bytes (includes time)	TEXT/INTEGER/REAL
TIME	3 bytes + fsp	8 bytes	3–5 bytes	— (part of DATE)	TEXT
DATETIME	5 bytes + fsp	—	8 bytes (DATETIME2 6–8)	7 bytes	TEXT
TIMESTAMP	4 bytes + fsp	8 bytes	8 bytes (rowversion, not a date)	7–11 bytes	TEXT
With zone	—	8 bytes (TIMESTAMPTZ)	8–10 (DATETIMEOFFSET)	13 bytes	—
Fractional seconds	DATETIME(3) / (6)	always microseconds	DATETIME2(3) / (7)	TIMESTAMP(6)	text

Table 4.2 — Bytes per value. fsp = fractional seconds precision (0–6, +0–3 bytes).

4.4 Instants versus wall clocks, decided properly

Ask one question about every temporal column: if a user in another time zone opens this row, should the displayed time change? If yes, it is an instant. If no, it is a wall clock. Then choose:

Need	Choose	Why not the alternative
Order placed at 14:03 UTC	TIMESTAMPTZ (PG), DATETIMEOFFSET (SQL Server), TIMESTAMP (MySQL)	A naive type records a number without its offset, so you cannot reconstruct the instant
Store opens at 09:00 local, every day	TIME or DATETIME, plus a separate zone column	An instant would shift when the local DST rules change
Invoice date for accounting	DATE	Accounting periods are date ranges in a declared zone, not instants
Flight departs 22:40 on 2026-03-01	TIMESTAMP WITH TIME ZONE + INTERVAL	The departure is an instant; the duration is not
Subscription length	INTERVAL, or months as INTEGER	Adding '1 month' is not adding 30 days
Ages and tenure	Integer days, or INTERVAL	Comparable, sortable, easy to bucket

Table 4.3 — The wall-clock/instant decision table.

`universal · one instant, three renderings`

```
-- The right way: instants in UTC, converted only for display
CREATE TABLE events (
    event_id   BIGINT PRIMARY KEY,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),   -- instant, stored UTC
    business_date DATE NOT NULL                        -- pre-computed in the reporting zone
);

-- PostgreSQL: the session zone decides how the instant is rendered
SET TIME ZONE 'Asia/Kolkata';
SELECT occurred_at FROM events;                    -- 2026-10-27 09:30:00+05:30
SET TIME ZONE 'UTC';
SELECT occurred_at FROM events;                    -- 2026-10-27 04:00:00+00

-- SQL Server
SELECT SWITCHOFFSET(occurred_at, '+05:30') FROM events;
SELECT occurred_at AT TIME ZONE 'India Standard Time' FROM events;

-- MySQL: TIMESTAMP converts using the session time_zone, DATETIME does not
SET time_zone = '+05:30';
SELECT created_at, created_local FROM orders;
```

GO DEEPER · Why TIMESTAMPTZ does not store a time zone
PostgreSQL's TIMESTAMPTZ normalises to UTC on write and has no memory of the input offset; it only formats using the session zone. If you need to know the original local offset (for example, which day the customer experienced the event), store it in a second column or use SQL Server's DATETIMEOFFSET, which keeps the offset.

4.5 Time zones, DST and the two-hour bug

- **Never store a local time and call it universal.** If the server is in UTC but the business is in Asia/Kolkata, a naive CURRENT_TIMESTAMP is the wrong wall clock and, worse, is inconsistent with application-generated times.
- **DST creates nonexistent and duplicated times.** On spring-forward, 02:30 may not exist; on fall-back it happens twice. A naive wall clock cannot distinguish them; an instant can.
- **Offsets are not zones.** +05:30 is a fixed offset; Asia/Kolkata is a zone with historical rule changes. Store zones (or instants) and convert with a zone database, not with a hard-coded offset.
- **Set the session zone explicitly in ETL.** A pipeline whose output depends on the server default is not reproducible; set `SET TIME ZONE 'UTC'` at the start of every job.
- **Beware the DATE at midnight trick.** Casting a timestamp to a date uses the session zone; if ETL runs in UTC and the business day is IST, every event between 18:30 and 24:00 UTC lands on the wrong day.

universal · converting before truncating

```
-- The 'wrong business date' bug, in full
-- Business day = Asia/Kolkata. ETL session = UTC.
SELECT occurred_at::date AS business_date FROM events;
-- 2026-10-27 20:00 IST = 2026-10-27 14:30 UTC → correct day
-- 2026-10-28 02:00 IST = 2026-10-27 20:30 UTC → counted on the 27th. Wrong by a day.

-- Fix: convert first, then cast
SELECT (occurred_at AT TIME ZONE 'Asia/Kolkata')::date AS business_date FROM events;

-- MySQL
SELECT DATE(CONVERT_TZ(occurred_at, '+00:00', '+05:30')) AS business_date FROM events;

-- SQL Server
SELECT CAST(occurred_at AT TIME ZONE 'UTC' AT TIME ZONE 'India Standard Time' AS date);
```

BEST PRACTICE · Store UTC, compute local, display local
Storage is UTC instants. Computations that need a calendar (business day, fiscal week, month-to-date) are done after converting to the business zone, and the resulting DATE is often materialised as its own column. Display converts at the last moment, in the presentation layer.

4.6 Ranges, precision and the 2038 problem

Engine	Type	Earliest	Latest	Notes
MySQL	DATETIME	1000-01-01	9999-12-31	No zone, 5 bytes + fsp
MySQL	TIMESTAMP	1970-01-01 00:00:01 UTC	2038-01-19 03:14:07 UTC	32-bit epoch, 4 bytes
PostgreSQL	TIMESTAMP / TIMESTAMPTZ	4713 BC	294276 AD	64-bit microseconds
SQL Server	DATETIME	1753-01-01	9999-12-31	3.33 ms accuracy
SQL Server	DATETIME2	0001-01-01	9999-12-31	100 ns; the default choice
SQL Server	SMALLDATETIME	1900-01-01	2079-06-06	1-minute accuracy
Oracle	DATE	4712 BC	9999 AD	7 bytes, includes HH:MM:SS
Oracle	TIMESTAMP(9)	4712 BC	9999 AD	7–11 bytes, nanoseconds
SQLite	TEXT/INTEGER/REAL	—	—	No type; conventions only

Table 4.4 — Ranges decide which types you can trust in 2038 and beyond.

WARNING · 2038 is a data type bug, not a Y2K rerun
MySQL TIMESTAMP and 32-bit Unix-time integers overflow on 2038-01-19. Any column holding an epoch in INT, any TIMESTAMP column with a 30-year mortgage or a pension schedule, and any future-dated record hits it. Use DATETIME, BIGINT epochs, or TIMESTAMPTZ.

Fractional seconds

Precision is declared and paid for. MySQL accepts DATETIME(0–6); SQL Server DATETIME2(0–7); Oracle TIMESTAMP(0–9); PostgreSQL always stores microseconds. A 3-digit millisecond default is usually right for application events; keep 6 digits for logs where ordering within a second matters.

universal · hour bucketing, four engines

```
CREATE TABLE ticks (
    at_ms  DATETIME(3)     NOT NULL,   -- MySQL: milliseconds, +2 bytes
    at_us  DATETIME(6)     NOT NULL    -- microseconds, +3 bytes
);

-- Truncate rather than round when bucketing time
SELECT DATE_FORMAT(at_ms, '%Y-%m-%d %H:00:00') AS hour_bucket FROM ticks;       -- MySQL
SELECT date_trunc('hour', at_us) AS hour_bucket FROM ticks;                    -- PostgreSQL
SELECT DATEADD(hour, DATEDIFF(hour, 0, at_us), 0) AS hour_bucket FROM ticks;   -- SQL Server
SELECT TRUNC(at_us, 'HH24') AS hour_bucket FROM ticks;                         -- Oracle
```

4.7 Date arithmetic and the functions worth memorising

Operation	MySQL	PostgreSQL	SQL Server	Oracle
Add 1 day	DATE_ADD(d, INTERVAL 1 DAY)	d + INTERVAL '1 day'	DATEADD(day, 1, d)	d + 1
Add 1 month	DATE_ADD(d, INTERVAL 1 MONTH)	d + INTERVAL '1 month'	DATEADD(month, 1, d)	ADD_MONTHS(d, 1)
Difference in days	DATEDIFF(a, b)	b - a (integer days)	DATEDIFF(day, a, b)	b - a
Current instant	NOW() / CURRENT_TIMESTAMP	now() / CURRENT_TIMESTAMP	SYSDATETIME()	SYSTIMESTAMP
Current UTC	UTC_TIMESTAMP()	now() AT TIME ZONE 'UTC'	SYSUTCDATETIME()	SYS_EXTRACT_UTC(SYSTIMESTAMP)
Truncate to month	DATE_FORMAT(d,'%Y-%m-01')	date_trunc('month', d)	DATETRUNC(month, d) (2022+)	TRUNC(d,'MM')
Extract part	EXTRACT(YEAR FROM d)	EXTRACT(YEAR FROM d)	YEAR(d) / DATEPART	EXTRACT(YEAR FROM d)
Last day of month	LAST_DAY(d)	date_trunc('month',d)+1mo-1day	EOMONTH(d)	LAST_DAY(d)
Format	DATE_FORMAT(d,'%d/%m/%Y')	to_char(d,'DD/MM/YYYY')	FORMAT(d,'dd/MM/yyyy')	TO_CHAR(d,'DD/MM/YYYY')
Parse text	STR_TO_DATE(s,fmt)	to_date(s, fmt)	CONVERT(d, 103)	TO_DATE(s, fmt)

Table 4.5 — Date arithmetic cheat sheet.

universal · age is harder than it looks

```sql
-- Age in completed years
-- PostgreSQL
SELECT date_part('year', age(CURRENT_DATE, birth_date)) FROM people;
-- MySQL
SELECT TIMESTAMPDIFF(YEAR, birth_date, CURDATE()) FROM people;
-- SQL Server
SELECT DATEDIFF(year, birth_date, GETDATE()) FROM people;   -- ⚠ counts year boundaries, not years
SELECT DATEDIFF(year, birth_date, GETDATE())
       - CASE WHEN DATEADD(year, DATEDIFF(year, birth_date, GETDATE()), birth_date) > GETDATE()
              THEN 1 ELSE 0 END AS age FROM people;
```

COMMON TRAP · BETWEEN on timestamps excludes the last day
WHERE created_at BETWEEN '2026-10-01' AND '2026-10-31' silently drops everything after midnight on the 31st. Use a half-open range: created_at >= '2026-10-01' AND created_at < '2026-11-01'. It is correct for timestamps, sargable, and safe with fractional seconds.

4.8 Generating calendars and bucketing time

universal · date spines prevent missing-month bugs

```sql
-- PostgreSQL: a date spine, the foundation of every time series report
SELECT d::date AS day
FROM generate_series('2026-01-01'::date, '2026-12-31'::date, '1 day') AS d;

-- MySQL 8: recursive CTE
WITH RECURSIVE days AS (
    SELECT DATE('2026-01-01') AS d
    UNION ALL
    SELECT d + INTERVAL 1 DAY FROM days WHERE d < '2026-12-31'
)
SELECT d FROM days;

-- SQL Server: system table or a numbers table
WITH days AS (
    SELECT DATEADD(day, n, '2026-01-01') AS d
    FROM (SELECT TOP (365) ROW_NUMBER() OVER (ORDER BY (SELECT 1)) - 1 AS n
```

```
          FROM sys.all_objects) n
)
SELECT d FROM days;

-- Monthly aggregation with a spine so empty months appear as zero
SELECT c.month, COALESCE(SUM(s.net_amount), 0) AS revenue
FROM calendar_months c
LEFT JOIN fact_sales s ON s.business_date >= c.month AND s.business_date < c.next_month
GROUP BY c.month
ORDER BY c.month;
```

TIP · The calendar dimension
Warehouses materialise a calendar table: date, day name, week, month, quarter, fiscal period, is_holiday, is_weekend. It makes reporting queries readable, keeps business logic out of CASE expressions, and gives every date type a single join target.

4.9 Temporal pitfalls checklist

Symptom	Cause	Fix
Report totals shift by one day	Casting timestamp to date in the wrong zone	Convert to the business zone, then cast
Last day of the month missing	BETWEEN with a date literal	Half-open range: >= start AND < next start
Two rows for one event	DST fall-back duplication	Store instants, not wall clocks
Times differ between app and database	Server default zone	Set the session zone explicitly; store UTC
“Invalid date” on insert	MySQL strict mode rejected a zero date	Use NULL, not '0000-00-00'
Age off by one	DATEDIFF counting boundaries	Compute with full date arithmetic
Index not used on a date filter	Function applied to the column	Filter on raw column; move functions to the constant
Future dates become 1970	32-bit TIMESTAMP overflow	DATETIME/TIMESTAMPTZ or BIGINT epoch

Table 4.6 — Eight temporal defects and their fixes.

4.10 Engine cheat sheet: temporal types

Need	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
Calendar date	DATE	DATE	DATE	DATE (with time)	TEXT
Naive date+time	DATETIME	TIMESTAMP	DATETIME2	TIMESTAMP	TEXT
Instant	TIMESTAMP (2038 limit)	TIMESTAMPTZ	DATETIMEOFFSET	TIMESTAMP WITH TIME ZONE	TEXT (UTC)
Time only	TIME	TIME	TIME	INTERVAL DAY TO SECOND	TEXT
Duration	(no type)	INTERVAL	(no type)	INTERVAL DAY TO SECOND	(no type)
Epoch seconds	INT / BIGINT	INTEGER / BIGINT	INT / BIGINT	NUMBER	INTEGER

Table 4.7 — Temporal types across engines.

TIP · Try it
In PostgreSQL, insert the same value as TIMESTAMP and TIMESTAMPTZ, then change the session zone and select both. Only one of them changes — that is the difference between a wall clock and an instant, demonstrated in ten seconds.

PART 5

Boolean, Binary & Identifiers

Two-state logic, raw bytes and the keys that identify a row

This part covers the types that are small in bytes and large in consequences: booleans that decide which rows exist, binary columns that hold hashes, keys and files, and identifier types where the choice between BIGINT and UUID shapes your index for years.

5.1 BOOLEAN and its four impersonations

Engine	True boolean	Common emulation	Notes
PostgreSQL	BOOLEAN	—	Accepts TRUE/FALSE, 't'/'f', 'yes'/'no', 1/0; prints t/f
MySQL	None (TINYINT(1))	TINYINT(1), ENUM('Y','N'), CHAR(1)	TRUE and FALSE are aliases for 1 and 0
SQL Server	BIT	CHAR(1), TINYINT	BIT stores 0, 1 or NULL; up to 8 BIT columns share a byte
Oracle	BOOLEAN (23ai)	NUMBER(1), CHAR(1)	Before 23ai, BOOLEAN existed only in PL/SQL
SQLite	None	INTEGER 0/1	Any value can be stored; CHECK (flag IN (0,1)) enforces it

Table 5.1 — Boolean support is the least portable thing in SQL.

universal · one flag, four declarations

```sql
-- PostgreSQL: a real boolean, with the natural syntax
CREATE TABLE users (is_active BOOLEAN NOT NULL DEFAULT TRUE);
SELECT * FROM users WHERE is_active;                  -- no = TRUE needed
SELECT * FROM users WHERE NOT is_active;

-- MySQL: TINYINT(1) by convention, 1/0 by necessity
CREATE TABLE users (is_active TINYINT(1) NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)));
SELECT * FROM users WHERE is_active = 1;

-- SQL Server: BIT
CREATE TABLE users (is_active BIT NOT NULL DEFAULT 1);
SELECT * FROM users WHERE is_active = 1;

-- Oracle 23ai: native BOOLEAN in SQL
CREATE TABLE users (is_active BOOLEAN DEFAULT TRUE);
```

COMMON TRAP · TINYINT(1) is not a boolean

In MySQL, the (1) is a display width: a TINYINT(1) column will happily store 42, -7 or 100. If the column means “yes or no”, add CHECK (col IN (0,1)) or use an ENUM. The width does not enforce anything and is deprecated in MySQL 8.0.17+.

Three-valued logic in practice

universal · flags have three states unless you forbid the third

```sql
-- With NULL flags, three states exist: TRUE, FALSE, UNKNOWN
SELECT COUNT(*) AS total,
       COUNT(*) FILTER (WHERE is_active)      AS active,        -- PostgreSQL
       COUNT(*) FILTER (WHERE NOT is_active)  AS inactive,
       COUNT(*) FILTER (WHERE is_active IS NULL) AS unknown
FROM users;
```

```
-- MySQL equivalent
SELECT SUM(is_active = 1), SUM(is_active = 0), SUM(is_active IS NULL) FROM users;

-- Dangerous: NOT IN with a nullable boolean
WHERE status NOT IN (SELECT flag FROM flags)   -- returns nothing if any flag is NULL

-- Safe: explicit exclusion of NULLs
WHERE status NOT IN (SELECT flag FROM flags WHERE flag IS NOT NULL)
```

BEST PRACTICE · Make flags NOT NULL
A nullable boolean has three meanings and every consumer guesses differently: the dashboard treats NULL as false, the API sends null, the filter drops the row. Default to NOT NULL with an explicit default, and only allow NULL when "unknown" is a genuine business state.

5.2 Binary types: BINARY, VARBINARY, BLOB, BYTEA, RAW

Type	Engine	Storage	Max size	Use for
BINARY(n)	MySQL, SQL Server	Fixed, zero-padded to n	255 / 8,000	Fixed hashes, fixed keys
VARBINARY(n)	MySQL, SQL Server	Variable + length prefix	65,535 / 8,000	Tokens, small payloads
VARBINARY(MAX)	SQL Server	Off-row above 8 KB	2 GB	Files, images, serialised objects
BLOB	MySQL, Oracle, SQLite	Off-page pointer + payload	64 KB – 4 GB (by variant)	Documents, images
BYTEA	PostgreSQL	Variable, TOAST-compressed	1 GB	Binary payloads of any kind
RAW(n)	Oracle	Fixed, not converted	2,000 bytes	Keys, hashes
IMAGE / LONG RAW	SQL Server / Oracle	Deprecated	—	Legacy; migrate to VARBINARY(MAX)/BLOB

Table 5.2 — Binary types and what they cost.

universal · the same hash, four literal syntaxes

```
-- Storing a SHA-256 hash: the fixed-width case
-- PostgreSQL
CREATE TABLE files (sha256 BYTEA NOT NULL CHECK (octet_length(sha256) = 32));
-- MySQL
CREATE TABLE files (sha256 BINARY(32) NOT NULL);
-- SQL Server
CREATE TABLE files (sha256 BINARY(32) NOT NULL);

-- Hex literals differ by engine
INSERT INTO files VALUES (DECODE('9f86d081...', 'hex'));   -- Oracle
INSERT INTO files VALUES (UNHEX('9f86d081...'));           -- MySQL
INSERT INTO files VALUES ('\x9f86d081...'::bytea);        -- PostgreSQL
INSERT INTO files VALUES (0x9F86D081...);                  -- SQL Server

-- Readable output
SELECT encode(sha256, 'hex') FROM files;   -- PostgreSQL
SELECT HEX(sha256) FROM files;             -- MySQL
SELECT CONVERT(VARCHAR(64), sha256, 2) FROM files;  -- SQL Server
```

- **Binary comparisons are byte-exact and case-sensitive** — unlike CHAR with a case-insensitive collation. That makes BINARY(32) the right home for hashes, tokens and checksums.
- **Zero padding surprises.** BINARY(n) pads with 0x00, so BINARY(4) 'AB' ≠ BINARY(2) 'AB'. Use VARBINARY unless the length is guaranteed.

- **Do not store large files in the database by default.** Blobs inflate backups, break streaming replication patterns, and are invisible to BI tools. Store them in object storage and keep the pointer in the row — unless transactional consistency with the row is a hard requirement.
- **Encryption and hashing live here.** Password hashes, PII tokenisation and column-level encryption payloads are all binary. Size the column for the algorithm's output, not for today's data.

5.3 Identifiers: BIGINT, UUID, ULID and natural keys

Key type	Bytes	Ordering	Insert locality	Portable	Best for
INT identity	4	Sequential	Excellent	Yes	Small tables, dimension keys
BIGINT identity	8	Sequential	Excellent	Yes	Fact tables, event streams
UUIDv4	16	Random	Poor (page splits)	Yes	Distributed writes, public IDs
UUIDv7 / ULID	16	Time-ordered	Good	Mostly	Distributed writes with range scans
Snowflake ID	8	Time-ordered	Good	Manual	High-volume distributed systems
Natural key (email)	20–254	Depends	Good but wide	Yes	Dimensions, with a surrogate PK
Composite key	Sum	Depends	Good	Yes	Junction tables, no surrogate needed

Table 5.3 — Keys are the most consequential type decision in a schema.

universal · UUIDs, and how to keep them from wrecking an index

```
-- PostgreSQL: native UUID, with generation strategies
CREATE TABLE orders (
    id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),      -- v4: random
-- id UUID PRIMARY KEY DEFAULT uuidv7(), -- PG 18+: time-ordered
    ref BIGINT GENERATED ALWAYS AS IDENTITY UNIQUE       -- human-friendly invoice number
);

-- SQL Server: UNIQUEIDENTIFIER, with a sequential default worth knowing
CREATE TABLE orders (
    id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID()
-- NEWID() is random and fragments the clustered index;
-- NEWSEQUENTIALID() is monotonic within the machine.
);

-- MySQL: UUID stored as BINARY(16) or as text
CREATE TABLE orders (
    id      BINARY(16) PRIMARY KEY,                      -- 16 bytes instead of 36
    id_text CHAR(36) GENERATED ALWAYS AS (
              INSERT(BIN_TO_UUID(id), 1, 1, '-')
            ) VIRTUAL
);
-- UUID_TO_BIN(uuid, 1) reorders the time fields for better index locality

-- Oracle
CREATE TABLE orders (id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY);
```

GO DEEPER · Why random UUIDs hurt

A clustered or B-tree index keeps keys in order. Random 16-byte keys insert into random leaf pages: more page splits, more I/O, a larger and more fragmented index. Time-ordered UUIDs (UUIDv7, ULID) and MySQL's UUID_TO_BIN(x, 1) restore locality while keeping global uniqueness. On PostgreSQL with a separate primary key index the effect is smaller but still measurable at scale.

BEST PRACTICE · The two-key pattern
Keep a compact surrogate key (BIGINT) for joins and storage, plus a natural or public key (UUID, order code, email) with a UNIQUE index for lookups and external references. Joins stay narrow, external identifiers stay opaque and safe to expose.

5.4 Special-purpose types you will meet in schemas

Type	Engine	What it does	When to use
ROWVERSION / TIMESTAMP	SQL Server	8-byte binary that changes on update	Optimistic concurrency, change detection
SQL_VARIANT	SQL Server	Holds any type in one column	Avoid: opaque, slow, unindexable in practice
SYS_GUID() / RAW(16)	Oracle	16-byte globally unique identifier	Surrogate keys without sequences
OID	PostgreSQL	Object identifier	Internal; do not design with it
HSTORE	PostgreSQL	Key-value text pairs	Sparse attributes; JSONB is usually better
VECTOR	pgvector, Oracle 23ai, MySQL 9	Embeddings of n dimensions	Similarity search over ML embeddings
INET / CIDR / MACADDR	PostgreSQL	Network addresses with validation	Logs, device inventories
GEOMETRY / GEOGRAPHY	All major engines	Points, lines, polygons with SRID	Maps, distance, catchment analysis
TSVECTOR	PostgreSQL	Pre-processed searchable text	Full-text indexes
DOMAIN / ENUM / COMPOSITE	PostgreSQL, MySQL (ENUM)	User-defined types	Centralise validation and reuse it

Table 5.4 — Types that look exotic and are routinely useful.

postgresql · domains, composites and network types

```postgresql
-- PostgreSQL: a domain is a reusable, checkable type
CREATE DOMAIN email_address AS TEXT
    CHECK (VALUE ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$');

CREATE TABLE customers (
    email email_address NOT NULL UNIQUE      -- validation travels with the type
);

-- A composite type for repeated structures
CREATE TYPE address AS (line1 TEXT, city TEXT, postal_code TEXT, country CHAR(2));
CREATE TABLE suppliers (id INT, billing address);

-- Network types reject bad data at insert time
CREATE TABLE devices (ip INET, network CIDR, hw MACADDR);
INSERT INTO devices VALUES ('192.168.1.10', '192.168.1.0/24', '08:00:2b:01:02:03');
```

PART 6

Structured & Special Types

JSON, XML, arrays, enums, ranges, spatial and vector columns

Everything so far was a single scalar value. This part covers types that hold **structure** inside one column — documents, arrays, key/value maps, enumerations, numeric ranges, geometries and embeddings. They are powerful, they are unevenly supported, and every one of them comes with a rule about when not to use it.

6.1 When a structured type is the right answer

Situation	Structured type?	Better option
Attributes that differ per product category	Yes — JSONB	A sparse attribute table, if you must query many of them
A fixed set of 12 form fields	No	Twelve ordinary columns
A list of tags per row	Yes — ARRAY or JSON	A junction table if you join or count by tag
A status with 5 known values	Yes — ENUM or CHECK	A lookup table if the list changes without a release
Saving an API response for audit	Yes — JSON	Nothing better; store verbatim, do not query deeply
Price bands (0–100, 100–500 …)	Yes — RANGE	Two columns plus a BETWEEN, if ranges are simple
Location of a store	Yes — spatial	Two decimal columns if you only ever display the point
A user's permissions	Rarely — bit mask	A roles table with a join

Table 6.1 — The structured-type decision table.

BEST PRACTICE · The first-normal-form rule

If a value is something you filter on, group by, join on or constrain, it deserves its own column. Structured types are for the long tail of attributes that are read as a whole, and for payloads you keep for completeness rather than analysis.

6.2 JSON and JSONB

Engine	Type	Storage	Indexable	Notes
PostgreSQL	JSON	Text, verbatim	No (expression indexes only)	Keeps whitespace, key order and duplicates
PostgreSQL	JSONB	Binary, parsed	Yes — GIN, expression	Deduplicates keys, sorts them; the default choice
MySQL 8	JSON	Binary (JSONB-like)	Yes — multi-valued and generated-column indexes	Validated on insert; 1 GB max
SQL Server	NVARCHAR(MAX) + ISJSON	Text	Yes — computed columns + indexes	No native JSON type; JSON functions over text
Oracle 23ai	JSON	Binary OSON	Yes — JSON search index	Native JSON type since 21c
SQLite	TEXT + json1	Text	Expression indexes only	Functions available since 3.38

Table 6.2 — JSON support is the most uneven feature in SQL.

`postgresql · jsonb from storage to query`

```sql
-- PostgreSQL: JSONB, the reference implementation
CREATE TABLE products (
    product_id BIGINT PRIMARY KEY,
    attributes JSONB NOT NULL DEFAULT '{}'::jsonb
);

INSERT INTO products VALUES
(1, '{"brand":"Acme","colour":"blue","sizes":[38,39,40],"price":{"amount":2499,"currency":"INR"}}');

-- Read a path (-> returns jsonb, ->> returns text)
SELECT attributes->>'brand'                          AS brand,
       (attributes->'price'->>'amount')::numeric     AS price
FROM products;

-- Filter with containment: does the document include this pair?
SELECT * FROM products WHERE attributes @> '{"colour":"blue"}';

-- Does a key exist?
SELECT * FROM products WHERE attributes ? 'sizes';

-- Expand an array into rows (the shape change analysts need most)
SELECT product_id, jsonb_array_elements_text(attributes->'sizes') AS size
FROM products;

-- Index for containment and existence queries
CREATE INDEX ix_products_attrs ON products USING GIN (attributes);
-- Index for one hot path
CREATE INDEX ix_products_brand ON products ((attributes->>'brand'));
```

engine-specific · JSON in MySQL, SQL Server and Oracle

```sql
-- MySQL 8: JSON paths use $ syntax
SELECT attributes->>'$.brand' AS brand,
       JSON_EXTRACT(attributes, '$.price.amount') AS price
FROM products;

SELECT * FROM products WHERE JSON_CONTAINS(attributes, '"blue"', '$.colour');

-- Index one extracted value with a generated column
ALTER TABLE products
  ADD COLUMN brand VARCHAR(60) GENERATED ALWAYS AS (attributes->>'$.brand') STORED,
  ADD INDEX ix_brand (brand);

-- Multi-valued index over an array (MySQL 8.0.17+)
ALTER TABLE products ADD INDEX ix_sizes ((CAST(attributes->'$.sizes' AS UNSIGNED ARRAY)));

-- SQL Server: JSON functions over NVARCHAR, with a validated check
CREATE TABLE products (
    product_id BIGINT PRIMARY KEY,
    attributes NVARCHAR(MAX) CHECK (ISJSON(attributes) = 1)
);
SELECT JSON_VALUE(attributes, '$.brand') FROM products;
SELECT * FROM products WHERE JSON_VALUE(attributes, '$.colour') = 'blue';

-- Oracle: dot notation on a native JSON column
SELECT p.attributes.brand.string() FROM products p;
```

- **JSONB rewrites your document**: keys sorted, duplicates dropped, whitespace gone. If you need the exact bytes for a digital signature, store text.

- **Every key is untyped.** A price written as "2499" (string) and 2499 (number) are different values and comparison silently fails. Validate at the boundary.
- **Indexes are opt-in.** Unlike a column index, a GIN index must be created and must match the operator you use. A JSON query without one is a full scan with parsing overhead.
- **Size matters.** Documents over a few kilobytes push the row off-page and inflate WAL. Keep the document for the sparse attributes, not for the big payload.
- **Consider generating columns** for the two or three paths you query constantly; that gives you ordinary index behaviour with JSON flexibility.

COMMON TRAP · JSON is not a schema-avoidance strategy
Teams adopt JSON to avoid migrations, then spend a year building validation in application code that a CHECK constraint would have done for free. Use JSON for genuine variability; use columns for the shape you know.

6.3 XML and other document types

Type	Engine	Storage	Use when
XML / XMLTYPE	SQL Server, Oracle, PostgreSQL (xml)	Parsed text, optional schema validation	Contractual feeds: ISO 20022, UBL invoices, SOAP
HSTORE	PostgreSQL	Binary key/value map	Rarely — JSONB covers the same ground with better functions
CLOB + XMLTYPE	Oracle	Character large object	Legacy migrations; prefer XMLTYPE for XQuery

Table 6.3 — XML is still the language of B2B contracts and banking messages.

universal · pulling a number out of XML

```
-- PostgreSQL: XPath and XPath extraction
CREATE TABLE invoices (id BIGINT, doc XML);
SELECT (xpath('/invoice/total/text()', doc))[1]::text::numeric AS total FROM invoices;

-- SQL Server: XML methods and typed columns
CREATE TABLE invoices (id BIGINT, doc XML);
SELECT doc.value('(/invoice/total)[1]', 'decimal(12,2)') AS total FROM invoices;

-- Oracle: extractValue, or XQuery with XMLTABLE for sets of rows
```

6.4 ARRAY types

universal · arrays exist, and levels of emulation below them

```
-- PostgreSQL: true array types, any element type
CREATE TABLE articles (
    id   BIGINT PRIMARY KEY,
    tags TEXT[] NOT NULL DEFAULT '{}'
);
INSERT INTO articles VALUES (1, ARRAY['sql','analytics']);
SELECT * FROM articles WHERE 'sql' = ANY(tags);        -- membership, index-friendly with GIN
SELECT id, unnest(tags) AS tag FROM articles;          -- one row per tag
CREATE INDEX ix_articles_tags ON articles USING GIN (tags);

-- Multi-dimensional and fixed-length arrays are allowed
SELECT ARRAY[[1,2],[3,4]] AS matrix;

-- MySQL 8: no array type; use JSON arrays
SELECT * FROM articles WHERE JSON_CONTAINS(tags, '"sql"');

-- SQL Server: no array type; use OPENJSON over an NVARCHAR payload
SELECT a.id, t.value AS tag
FROM articles a CROSS APPLY OPENJSON(a.tags) AS t;
```

```sql
-- SQLite: json_each
SELECT a.id, j.value AS tag FROM articles a, json_each(a.tags) j;

-- Oracle: nested tables / VARRAY in PL/SQL; in SQL, use JSON arrays
```

WARNING · Arrays break the relational rules you rely on

You cannot foreign-key an array element, enforce uniqueness inside an array, or get a cheap count per tag without unnesting. If tags are a dimension you report on, model them as a junction table; keep the array only for display in a single row.

6.5 ENUM and SET

mysql & postgresql · enums, three ways

```sql
-- MySQL: ENUM is a column property with a fixed list
CREATE TABLE orders (
    id     BIGINT PRIMARY KEY,
    status ENUM('new','paid','shipped','cancelled') NOT NULL DEFAULT 'new'
);
-- Storage: 1 byte for ≤255 values, 2 bytes for ≤65,535
SELECT status, status + 0 AS ordinal FROM orders;   -- ordinals are 1-based and fragile
-- Adding a value rewrites the table on older versions; MySQL 8 appends in place at the end

-- MySQL SET: up to 64 values in a bit mask
CREATE TABLE prefs (channels SET('email','sms','push'));
INSERT INTO prefs VALUES ('email,push');

-- PostgreSQL: enum is a real type you can reuse
CREATE TYPE order_status AS ENUM ('new','paid','shipped','cancelled');
CREATE TABLE orders (id BIGINT PRIMARY KEY, status order_status NOT NULL DEFAULT 'new');
ALTER TYPE order_status ADD VALUE 'refunded';      -- works, but not inside a transaction block

-- Portable alternative: VARCHAR + CHECK (or a lookup table)
status VARCHAR(12) NOT NULL CHECK (status IN ('new','paid','shipped','cancelled'))
```

Approach	Pros	Cons
ENUM	Compact (1–2 bytes), validated, readable in queries	Changing the list needs DDL; ordering is by ordinal; hard to port
VARCHAR + CHECK	Portable, obvious, easy to change with a migration	Uses more bytes; the CHECK must be maintained
Lookup table + FK	Values change without DDL, can carry labels and sort order	Requires a join; slightly more complex ETL
Integer status + code table	Fastest joins, stable numbers	Unreadable in raw queries; mapping always needed

Table 6.4 — Four ways to constrain a status, and their trade-offs.

6.6 RANGE and MULTIRANGE (PostgreSQL)

postgresql · ranges with exclusion constraints

```sql
-- Built-in range types: int4range, int8range, numrange, daterange, tsrange, tstzrange
CREATE TABLE bookings (
    id     BIGINT PRIMARY KEY,
    room   INT NOT NULL,
    during TSTZRANGE NOT NULL,
    EXCLUDE USING GIST (room WITH =, during WITH &&)   -- no overlaps, enforced by the DB
);

INSERT INTO bookings VALUES (1, 101, '[2026-10-27 09:00, 2026-10-27 11:00)');
```

```
-- Range operators
SELECT * FROM bookings WHERE during @> '2026-10-27 10:00+05:30'::timestamptz;  -- contains
SELECT * FROM bookings WHERE during && '[2026-10-27, 2026-10-28)'::tstzrange;  -- overlaps
SELECT upper(during) - lower(during) AS duration FROM bookings;

-- A range index makes overlap searches fast
CREATE INDEX ix_bookings_during ON bookings USING GIST (during);
```

GO DEEPER · Why ranges beat two columns

Two columns (valid_from, valid_to) express a range but cannot be indexed as a single interval, cannot be excluded from overlapping declaratively, and leave the boundary convention (inclusive or not) ambiguous. A range type plus a GiST index plus an EXCLUDE constraint enforces non-overlap in the database — a whole class of scheduling bugs disappears.

6.7 Spatial types

Engine	Types	Coordinates	Index
PostgreSQL + PostGIS	GEOMETRY, GEOGRAPHY	Plane (SRID) and spheroid	GiST
MySQL 8	GEOMETRY, POINT, LINESTRING, POLYGON + collection types	SRID-aware	SPATIAL (R-tree)
SQL Server	GEOMETRY, GEOGRAPHY	Plane and spheroid	SPATIAL
Oracle	SDO_GEOMETRY	SRID-aware	Spatial index (R-tree)
SQLite	SpatiaLite extension; or TEXT/REAL	Depends	R-tree virtual table

Table 6.5 — Spatial support by engine.

postgresql · geography in metres, geometry in degrees

```
-- PostgreSQL + PostGIS: distance and containment
CREATE TABLE stores (id INT PRIMARY KEY, name TEXT, location GEOGRAPHY(POINT, 4326));
INSERT INTO stores VALUES (1, 'Bandra', ST_MakePoint(72.8401, 19.0606)::geography);

SELECT name, ST_Distance(location, ST_MakePoint(72.8777,19.0760)::geography) / 1000 AS km
FROM stores ORDER BY km LIMIT 5;

-- Stores inside a service polygon
SELECT s.name FROM stores s
JOIN service_areas a ON ST_Contains(a.area, s.location::geometry)
WHERE a.city = 'Mumbai';

CREATE INDEX ix_stores_geo ON stores USING GIST (location);
```

6.8 Vectors and embeddings

Vector columns arrived because similarity search could not be expressed as a join. A vector is a fixed length array of floats; the index is built for approximate nearest-neighbour search.

postgresql / oracle · vector columns and nearest-neighbour queries

```
-- PostgreSQL with pgvector
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE docs (
    id        BIGINT PRIMARY KEY,
    content   TEXT,
    embedding VECTOR(1536)          -- dimension must match your model exactly
);
CREATE INDEX ix_docs_embedding ON docs USING hnsw (embedding vector_cosine_ops);

SELECT id, content, 1 - (embedding <=> '[0.01,0.02,...]'::vector) AS similarity
```

```
FROM docs ORDER BY embedding <=> '[...]'::vector LIMIT 5;

-- Oracle 23ai
CREATE TABLE docs (id NUMBER PRIMARY KEY, embedding VECTOR(1536, FLOAT32));
-- MySQL 9 / SQL Server 2025 also ship VECTOR types
```

NOTE · Vectors are a schema decision too

The dimension is part of the type: change the embedding model and you need a new column and a re-embedding job. Store the model name and version beside the vector, and keep the distance metric consistent with the one the model was trained for.

6.9 User-defined types: domains, composites, extensions

universal · reuse a rule instead of repeating a CHECK

```
-- PostgreSQL domains: validation that follows the type everywhere
CREATE DOMAIN positive_int AS INTEGER CHECK (VALUE > 0);
CREATE DOMAIN iso_country  AS CHAR(2) CHECK (VALUE ~ '^[A-Z]{2}$');

CREATE TABLE shipment (
    id       BIGINT PRIMARY KEY DEFAULT gen_random_uuid(),
    quantity positive_int NOT NULL,
    origin   iso_country  NOT NULL
);

-- Changing the rule once updates every column that uses the domain
ALTER DOMAIN positive_int ADD CONSTRAINT no_huge CHECK (VALUE < 100000);

-- Oracle: object types and nested tables
CREATE TYPE address_t AS OBJECT (line1 VARCHAR2(100), city VARCHAR2(50), country CHAR(2));

-- SQL Server: user-defined types are aliases, not constraints
CREATE TYPE phone FROM VARCHAR(20) NOT NULL;
```

6.10 Special-type pitfalls

Pitfall	Consequence	Mitigation
JSON queried without an index	Full scan and parse on every request	GIN index, generated column, or promote the key to a column
Mixed types inside JSON	"1" vs 1, silent no-match	Validate with a JSON schema at ingest
ENUM used for a changing list	DDL per new value, ordering surprises	Lookup table with a foreign key
Array instead of a junction table	Cannot join, count or constrain per element	Model the many-to-many properly
Spatial column without a spatial index	Distance queries scan everything	GIST/SPATIAL index and SRID consistency
Vector of the wrong dimension	Insert fails, or worse, silently truncated	Assert the dimension in the type and in tests
Domain changed without validation	Old rows violate the new rule	ALTER DOMAIN ... VALIDATE CONSTRAINT

Table 6.6 — Seven ways structured types go wrong.

PART 7

Casting & Storage Mechanics

Conversion rules, sargability, row layout and what a type change costs

Two columns with different types can still be comparable — and that is the problem. This part covers how engines convert values, which conversions are silent, which ones quietly disable an index, and how the bytes of a row are actually laid out on a page.

7.1 Implicit and explicit conversion

Kind	Example	Happens when	Risk
Implicit	WHERE varchar_col = 42	The engine must compare different types	Index disabled, wrong result, silent truncation
Explicit	CAST(x AS DECIMAL(10,2))	You wrote the cast	Visible; still can truncate or error
Assignment	INSERT INTO t(int_col) VALUES ('12x')	Value converted to the column type	MySQL non-strict: stores 12; strict: error
Try-conversion	TRY_CAST(x AS INT)	You asked for NULL on failure	Hides data problems unless logged

Table 7.1 — Four ways a value changes type.

universal · casting syntax

```
-- The cast syntax of five engines
SELECT CAST('2026-10-27' AS DATE);        -- standard SQL, everywhere

-- PostgreSQL: :: shorthand plus rich casting
SELECT '2026-10-27'::date, '12.5'::numeric(6,2), 42::text;

-- MySQL: CAST types are limited (SIGNED, UNSIGNED, DECIMAL, CHAR, DATE, DATETIME, TIME, JSON)
SELECT CAST('42' AS SIGNED), CAST(42 AS CHAR), CONVERT('12.5', DECIMAL(6,2));

-- SQL Server: CONVERT adds style numbers for dates
SELECT CONVERT(DATE, '27/10/2026', 103);      -- 103 = dd/mm/yyyy
SELECT TRY_CONVERT(INT, 'abc');                -- NULL instead of an error

-- Oracle: TO_DATE / TO_NUMBER / TO_CHAR with explicit formats
SELECT TO_DATE('27-10-2026','DD-MM-YYYY'), TO_NUMBER('1,234.5','9G999D9') FROM dual;
```

Conversion precedence

When types disagree, engines rank them and convert the lower-ranked side. SQL Server's precedence (simplified): sql_variant > datetimeoffset > datetime2 > datetime > smalldatetime > date > time > float > decimal > money > bigint > int > smallint > tinyint > bit > nvarchar > varchar. The practical consequence: comparing an INT column to an NVARCHAR parameter converts the **column** to NVARCHAR, and the index on it cannot be seeked.

sql server · implicit conversion is the classic performance bug

```
-- SQL Server: the implicit conversion that kills an index
-- orders.order_id is INT; @id is passed as NVARCHAR by the driver
SELECT * FROM orders WHERE order_id = @id;
-- Execution plan: CONVERT_IMPLICIT(int, @id, 0) applied to every row → scan

-- Fix: make the parameter the same type as the column
EXEC sp_executesql N'SELECT * FROM orders WHERE order_id = @id', N'@id INT', @id = 42;
```

```
-- Or catch it in advance
SELECT t.name AS table_name, c.name AS column_name, s.name AS statistics_name
FROM sys.stats s
JOIN sys.stats_columns sc ON s.object_id = sc.object_id AND s.stats_id = sc.stats_id
JOIN sys.columns c ON c.object_id = sc.object_id AND c.column_id = sc.column_id
JOIN sys.tables t ON t.object_id = s.object_id
WHERE s.user_created = 1;
```

7.2 Sargability: what makes an index usable

Predicate	Index usable?	Rewrite as
col = 42	Yes	—
col > 100 AND col < 200	Yes (range)	—
col LIKE 'abc%'	Yes	—
col LIKE '%abc'	No	Full-text or trigram index
CAST(col AS date) = '2026-10-27'	No	col >= '2026-10-27' AND col < '2026-10-28'
col + 0 = 42	No	col = 42
UPPER(col) = 'ABC'	No	Expression index on UPPER(col)
int_col = '42' (SQL Server)	Maybe (implicit convert)	Match the parameter type
COALESCE(col, 'x') = 'x'	No	col IS NULL OR col = 'x'
col IS NOT NULL	Rarely useful	Add a filtered index if needed
a.col = b.col	Hash/merge join	Ensure both sides share the same type

Table 7.2 — Rewrites that restore an index seek.

BEST PRACTICE · The three-second sargability check
Look at every WHERE and JOIN: is there a function, arithmetic or a cast on the column? If yes, move it to the constant side, or create an expression/functional index. No other single change improves query performance as often.

7.3 Type conversion tables worth knowing

From → To	Behaviour	Example
DECIMAL → INT	Rounds (PostgreSQL, MySQL) or truncates (SQL Server CAST)	CAST(2.7 AS INT): 3 on PG, 2 on SQL Server
FLOAT → DECIMAL	Takes the decimal expansion of the binary value	CAST(0.1::float8 AS numeric) = 0.1000000000000000055511151231257827
TEXT → DATE	Parsed per engine; ISO format is the only safe one	'27-10-2026' fails without a format mask
DATE → TEXT	Formatted using the session's default	Never rely on the default in ETL output
TIMESTAMP → DATE	Truncates, using the session zone	The DST day-shift bug from Part 4
BIGINT → INT	Overflow error (strict) or wrap (some engines)	Always check the maximum first
JSONB → TEXT	Serialised, keys reordered	Not byte-identical to the input
CHAR → VARCHAR	Trailing space handling differs	TRIM on write if it matters
UNSIGNED → SIGNED (MySQL)	Reinterprets the bits unless cast	CAST(u AS SIGNED) not needed in comparisons, but in arithmetic

Table 7.3 — Conversions that surprise people.

universal · never cast unvalidated text with a plain CAST

```
-- Safe conversion pattern for messy source data
-- PostgreSQL
SELECT CASE WHEN raw_value ~ '^[0-9]+$' THEN raw_value::int END AS parsed FROM staging;
```

```
-- SQL Server
SELECT TRY_CAST(raw_value AS INT) AS parsed,
       CASE WHEN TRY_CAST(raw_value AS INT) IS NULL THEN raw_value END AS rejected
FROM staging;

-- MySQL 8
SELECT REGEXP_REPLACE(raw_value, '[^0-9]', '') + 0 AS digits_only FROM staging;

-- Oracle
SELECT TO_NUMBER(REGEXP_SUBSTR(raw_value, '^[0-9]+$')) FROM staging;
```

WARNING · Casting in a pipeline is a data-quality signal
Every value that fails a cast is a defect upstream. Log rejected rows to a quarantine table with the reason, alert on the count, and never use TRY_CAST to silently discard them — that is how row counts quietly drop between source and warehouse.

7.4 Storage mechanics: how a row is laid out

Rows live in fixed-size pages (8 KB in SQL Server and PostgreSQL, 16 KB in InnoDB, 8 KB in Oracle). A page holds a header, item pointers and rows. Two details decide how many rows fit: **nullability** and **alignment padding**.

- **NULL bitmap.** A bitmap (1 bit per nullable column) marks which columns are NULL. In SQL Server and PostgreSQL it is present only if the table has nullable columns — a table of NOT NULL columns saves a byte or two per row and makes some operations cheaper.
- **Alignment.** Fixed-width columns are padded to their alignment (2, 4 or 8 bytes). Ordering columns in a row can reduce padding, but most engines reorder internally, so micro-optimising the order rarely helps.
- **Off-page storage.** Large values move to overflow/TOAST pages and leave a pointer: 20 bytes in PostgreSQL, or a 24-byte pointer in SQL Server row-overflow.
- **Variable-length updates.** Growing a VARCHAR can force the row to move to another page (forwarded records in SQL Server, row migration in InnoDB), which is why updates are slower than inserts.
- **Compression.** Page or row compression can halve the footprint of a wide table, but only for read-mostly data — and only if the types compress well (integers and dates compress far better than already-compressed binary).

universal · measure the row, do not guess it

```
-- Measuring real row cost
-- PostgreSQL
SELECT pg_size_pretty(pg_relation_size('fact_sales')) AS table_size,
       pg_size_pretty(pg_indexes_size('fact_sales'))  AS index_size,
       pg_size_pretty(pg_total_relation_size('fact_sales')) AS total;

SELECT avg_width, sum(avg_width) OVER () AS row_width_estimate
FROM pg_stats WHERE tablename = 'fact_sales' ORDER BY avg_width DESC;

-- MySQL
SELECT table_name, data_length, index_length, avg_row_length, table_rows
FROM information_schema.tables WHERE table_schema = DATABASE();

-- SQL Server
EXEC sp_spaceused 'fact_sales';
SELECT * FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID('fact_sales'), NULL, NULL, 'DETAILED');
```

7.5 What a type change costs

Change	Cost	Can it be done online?
Widen VARCHAR(50) → (100)	Metadata only on PostgreSQL/SQL Server; table rebuild on older MySQL	Usually yes
INT → BIGINT	Rebuild table + every index on the column, and every referencing FK	Rarely; needs a maintenance window
DATETIME → TIMESTAMPTZ	Rebuild, and the interpretation of every row changes	No; needs a data migration plan
FLOAT → DECIMAL	Rebuild plus rounding decisions per row	No; values change
VARCHAR → TEXT	Metadata only on PostgreSQL; rebuild or off-page on MySQL	Depends
Add a CHECK constraint	Scan to validate (unless NOT VALID / NOVALIDATE)	Yes, validate later
Change charset lorem → utf8mb4	Full rebuild, 1–4× disk during the operation	Only with online DDL tooling
Reorder or drop a column	Rebuild on most engines	No

Table 7.4 — Some type changes are metadata, others are migrations.

universal · making a type change survivable

```
-- PostgreSQL: add a constraint without blocking writes
ALTER TABLE orders ADD CONSTRAINT positive_amount CHECK (amount > 0) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT positive_amount;      -- takes a lighter lock

-- Oracle equivalent
ALTER TABLE orders ADD CONSTRAINT positive_amount CHECK (amount > 0) ENABLE NOVALIDATE;

-- MySQL 8: online DDL, and what it still cannot do in place
ALTER TABLE orders MODIFY amount DECIMAL(12,2), ALGORITHM=INPLACE, LOCK=NONE;
-- widening INT → BIGINT still rebuilds; use gh-ost / pt-online-schema-change

-- SQL Server: an online index rebuild for the dependent index
ALTER INDEX ALL ON orders REBUILD WITH (ONLINE = ON);
```

7.6 Partitioning and clustering follow the type

- **Partition on dates, not on text.** A DATE/TIMESTAMP key enables range pruning and rolling windows; a VARCHAR date prevents both.
- **Clustered index order is physical order.** In SQL Server, a random UNIQUEIDENTIFIER clustered key fragments the table; in InnoDB the primary key is the table, so a wide key widens every secondary index.
- **Hash partitioning** on an INTEGER or UUID spreads writes but makes range queries scan every partition. Pick the partition key from the dominant filter, not from the primary key.
- **Statistics depend on type.** Low-cardinality columns (booleans, small enums) make poor histograms; the optimiser may ignore them entirely. That is fine for filters, bad for joins.

7.7 Case study: a 40× slower report, fixed by two type changes

A daily revenue report took 4 minutes 20 seconds. The SQL was unchanged for a year; the table had grown from 2 million to 90 million rows. The execution plan showed two sequential scans and a nested loop with 90 million iterations.

Finding	Cause	Fix	Effect
Seq scan despite an index on sale_date	`WHERE sale_date::text LIKE '2026-10%'` — a cast on the column	Range predicate on the raw column	Index scan, 3 seconds
Join hash spilled to disk	customer_id INT joined to a VARCHAR(20) natural key	Join on the INTEGER surrogate key	Hash join in memory
Nested loop over 90 M rows	Implicit cast from a VARCHAR parameter to an INT column	Typed parameter, same type as the column	Nested loop became an index seek
Report still slow at month end	No partition pruning; one 90 M row table	Range partition by month on business_date	Scans only the month
Totals disagreed with finance	FLOAT amounts	DECIMAL(14,2) net_amount	Cent-level agreement

Table 7.5 — Every fix is a type decision, not a query trick.

TIP · The order to optimise

1) Fix the types. 2) Remove functions and casts from predicates. 3) Add the right index. 4) Partition. 5) Only then consider rewriting the query or adding a cache. Most teams start at step 5.

NOTES · PART 7 · CASTING & STORAGE

PART 8

Types Across Engines

The portability matrix: one concept, five spellings, zero surprises

This is the part you keep open during a migration. For every concept you need, it gives the type in each engine, the storage cost, and the note that prevents a broken migration. Read the three rules first — they resolve most questions without a table.

8.1 Three rules for mapping types

1 · Map the concept, not the name
DECIMAL, NUMERIC, NUMBER and MONEY are not interchangeable: MONEY fixes four decimals and rounds differently. Ask what the value means, then pick the destination type that expresses it.

2 · Widen, never narrow
If the source can hold 10 digits, the target must hold at least 10. VARCHAR(255) → VARCHAR(50) is a silent truncation waiting for a long value; INT → BIGINT is safe, the reverse is not.

3 · Text is not a universal adaptation layer
Mapping every unknown type to TEXT 'for safety' pushes conversion errors into every downstream query. Store the closest native type, and quarantine the rows that do not convert.

NOTE · What this part covers
MySQL 8.0/8.4, PostgreSQL 16/17, SQL Server 2022, Oracle 23ai and SQLite 3. Newer versions add types (native JSON, VECTOR, native BOOLEAN in Oracle); the rows below are the stable core.

8.2 Numeric mapping

Concept	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
Boolean-ish 1-byte	TINYINT UNSIGNED	SMALLINT	TINYINT	NUMBER(3)	INTEGER
Small integer	SMALLINT	SMALLINT	SMALLINT	NUMBER(5)	INTEGER
Integer	INT	INTEGER	INT	NUMBER(10)	INTEGER
Large integer	BIGINT	BIGINT	BIGINT	NUMBER(19)	INTEGER
Exact decimal p,s	DECIMAL(p,s)	NUMERIC(p,s)	DECIMAL(p,s)	NUMBER(p,s)	NUMERIC
4-byte float	FLOAT	REAL	REAL	BINARY_FLOAT	REAL
8-byte float	DOUBLE	DOUBLE PRECISION	FLOAT	BINARY_DOUBLE	REAL
Currency	DECIMAL(19,4)	NUMERIC(19,4)	DECIMAL(19,4) or MONEY	NUMBER(19,4)	INTEGER minor units
Auto increment	AUTO_INCREMENT	GENERATED AS IDENTITY	IDENTITY(1,1)	GENERATED AS IDENTITY	INTEGER PRIMARY KEY

Table 8.1 — Numeric concepts to concrete declarations.

- Oracle has no integer types: NUMBER(p,0) is the idiom, and its storage is variable, so NUMBER(10) is often smaller on disk than an INT elsewhere.
- SQL Server has no UNSIGNED. A MySQL `INT UNSIGNED` must become `BIGINT` to preserve the full 0–4.29 billion range.
- SQLite's NUMERIC affinity stores integers exactly and decimals as REAL — if exact money matters, store integer minor units.

8.3 Character and binary mapping

Concept	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
Fixed-length text	CHAR(n)	CHAR(n)	CHAR(n)/NCHAR(n)	CHAR(n)	TEXT
Variable text	VARCHAR(n)	VARCHAR(n)	VARCHAR(n)	VARCHAR2(n)	TEXT
Unicode text	VARCHAR(n) utf8mb4	VARCHAR(n) (UTF-8 db)	NVARCHAR(n)	NVARCHAR2(n)	TEXT (UTF-8)
Long text	LONGTEXT	TEXT	NVARCHAR(MAX)	CLOB / NCLOB	TEXT
Case-insensitive	COLLATE ..._ai_ci	citext	COLLATE ..._CI	NLS_SORT / LOWER index	COLLATE NOCASE
Fixed-length binary	BINARY(n)	BYTEA (no fixed)	BINARY(n)	RAW(n)	BLOB
Variable binary	VARBINARY(n)	BYTEA	VARBINARY(n)	RAW(n) / BLOB	BLOB
Large binary	LONGBLOB	BYTEA	VARBINARY(MAX)	BLOB	BLOB

Table 8.2 — Text and binary concepts.

WARNING · The Unicode trap in every migration to SQL Server
A MySQL utf8mb4 VARCHAR(255) mapped to VARCHAR(255) drops non-Latin text. It must become NVARCHAR(255) — and NVARCHAR counts characters in units of 2 bytes, so the 8,000-byte row limit means NVARCHAR(4,000) at most. Size the destination for bytes, not for characters.

8.4 Temporal mapping

Concept	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
Calendar date	DATE	DATE	DATE	DATE (has time)	TEXT or INTEGER
Wall-clock date+time	DATETIME(6)	TIMESTAMP	DATETIME2(7)	TIMESTAMP	TEXT
Instant	TIMESTAMP (2038 cap)	TIMESTAMPTZ	DATETIMEOFFSET	TIMESTAMP WITH TIME ZONE	TEXT (UTC)
Time of day	TIME	TIME	TIME	INTERVAL DS / DATE	TEXT
Duration	BIGINT seconds	INTERVAL	BIGINT seconds	INTERVAL DAY TO SECOND	INTEGER
Year only	YEAR / SMALLINT	SMALLINT or DATE	SMALLINT	NUMBER(4)	INTEGER
Milliseconds	DATETIME(3)	TIMESTAMP (always µs)	DATETIME2(3)	TIMESTAMP(3)	INTEGER epoch

Table 8.3 — Temporal concepts. Oracle's DATE includes a time, and has no sub-second part.

COMMON TRAP · Oracle DATE is not a date
Oracle DATE stores year, month, day, hour, minute and second in 7 bytes — no fractional seconds and no time zone. Migrating a PostgreSQL DATE to Oracle DATE is safe; migrating a TIMESTAMP(6) to Oracle DATE loses the microseconds, silently.

8.5 Identifiers and structured mapping

Concept	MySQL 8	PostgreSQL 16	SQL Server	Oracle 23ai	SQLite
UUID	BINARY(16) / CHAR(36)	UUID	UNIQUEIDENTIFIER	RAW(16)	TEXT / BLOB
JSON document	JSON	JSONB	NVARCHAR(MAX) + ISJSON	JSON	TEXT + json1
XML	No native (TEXT)	XML	XML	XMLTYPE	TEXT
Array	JSON array	native array	JSON + OPENJSON	JSON array / nested table	JSON
Enum	ENUM	CREATE TYPE ... AS ENUM	CHECK constraint	CHECK or lookup table	CHECK constraint
Range	No (2 columns)	int4range, daterange, tstzrange	No	No	No
Spatial	GEOMETRY + SRID	GEOMETRY/GEOGRAPHY + GiST	GEOMETRY/GEOGRAPHY	SDO_GEOMETRY	SpatiaLite
Vector	VECTOR (9+)	pgvector VECTOR(n)	VECTOR (2025)	VECTOR(n)	sqlite-vec
Special/other	SET, BIT	INET, MACADDR, TSVECTOR, HSTORE, domain	ROWVERSION, SQL_VARIANT	Object types	dynamic typing

Table 8.4 — Everything else, and the cheapest safe substitute.

8.6 SQLite: five affinities, no data types

SQLite has **storage classes** (NULL, INTEGER, REAL, TEXT, BLOB) and gives each declared type an **affinity** that guides conversion. A column declared DATE can hold the text 'hello'. That makes SQLite a poor authority on your schema — and a good reminder that in every engine the declared type is a promise the database may or may not keep.

Declared type contains	Affinity	How values are stored	Example
INT	INTEGER	Integer if convertible, else as given	INT, BIGINT, TINYINT
CHAR, CLOB, TEXT	TEXT	Text	VARCHAR(20), NVARCHAR, TEXT
BLOB or empty	BLOB	No conversion at all	BLOB, or a column with no type
REAL, FLOA, DOUB	REAL	Floating point	REAL, DOUBLE, FLOAT
Anything else (DECIMAL, DATE, BOOLEAN)	NUMERIC	Integer or real when lossless, else text	DECIMAL(10,2), BOOLEAN, DATE

Table 8.5 — The five SQLite affinities that swallow every other type name.

sqlite · affinity in action, and how to defend against it

```
CREATE TABLE t (d DATE);           -- NUMERIC affinity, so '2026-10-27' stays TEXT
INSERT INTO t VALUES ('2026-10-27');    -- stored as text
INSERT INTO t VALUES (20261027);         -- stored as integer
SELECT typeof(d) FROM t;                 -- 'text' then 'integer' — both legal

-- STRICT tables (SQLite 3.37+) restore a real type system
CREATE TABLE t2 (id INTEGER PRIMARY KEY, d TEXT NOT NULL) STRICT;

-- Conventions for a portable SQLite schema
-- dates : TEXT 'YYYY-MM-DD'
-- instants: TEXT ISO-8601 UTC with 'Z'
-- booleans: INTEGER 0/1 + CHECK
-- money : INTEGER minor units
```

8.7 Migration pitfalls by direction

Migration	Pitfall	Do this
MySQL → PostgreSQL	TINYINT(1) booleans become 0/1 integers; DATETIME has no zone	Map to BOOLEAN and TIMESTAMPTZ explicitly; test every boolean predicate
MySQL → SQL Server	utf8mb4 text truncated by VARCHAR; UNSIGNED range overflow	NVARCHAR(n) sized in bytes; BIGINT for unsigned INT
Oracle → PostgreSQL	NUMBER(38) with no scale becomes NUMERIC; DATE carries a time; empty string is NULL in Oracle	Choose precision per column; split DATE into DATE + TIME; audit '' → NULL
SQL Server → PostgreSQL	NVARCHAR → VARCHAR (fine, UTF-8); DATETIMEOFFSET → TIMESTAMPTZ; MONEY → NUMERIC	Verify offsets; MONEY needs a DECIMAL(19,4) decision, not a cast
PostgreSQL → MySQL	Arrays, JSONB operator set, ranges, domains have no direct equivalent	Flatten arrays to junction tables; rewrite @> and ? operators as JSON_CONTAINS/JSON_EXTRACT
Any → SQLite	Everything becomes affinity; NUMERIC may store decimals as REAL	Use STRICT tables and integer minor units for money
Any → BigQuery/Snowflake	No unsigned, no ENUM, NUMERIC precision caps (38), TIMESTAMP is UTC-only	Check precision caps first; characterise each numeric column

Table 8.6 — Seven migrations and the trap that waits in each.

8.8 The pre-migration checklist

1. Inventory every column with its type, precision, scale, length, nullability and default in the source.
2. Classify each into a family, then apply the tables above — never map name to name.
3. Compute the maximum length, precision and range actually present in the data: SELECT MAX(LENGTH(col)), MAX(col), MIN(col) per column.
4. Identify the time columns and decide instant versus wall clock for each one, explicitly.
5. Check charset coverage: scan for characters outside the target encoding before the migration, not after.
6. Mock-load 1% of the data and compare row counts, sums and min/max per column between source and target.
7. Verify booleans, zero dates, empty strings vs NULL, and 2038-dated values one by one.
8. Keep a mapping document in the repository — the next migration starts from it.

BEST PRACTICE · Automate the type check
Write a script that reads information_schema on both sides and diffs the type families, precision and nullability. A 60-line query catches the truncation, the unsigned overflow and the time-zone loss before a single row reaches production.

NOTES · PART 8 · ENGINE MAPPING

PART 9

Cheat Sheet & Practice

One-page reference, 15 exercises with answers, and a glossary

Everything in this guide, compressed. Print this part first: the cheat sheet answers 90% of everyday questions, the exercises tell you whether you actually understood Parts 1–8, and the glossary gives you the words to argue with a DBA.

9.1 The one-page cheat sheet

If you need…	Use this	Never this
Money, tax, totals	DECIMAL(19,4) or DECIMAL(12,2) + CHAR(3) currency	FLOAT, DOUBLE, MONEY
A counter or ID	BIGINT (INT only if the table stays under 2.1 B rows)	TINYINT, SMALLINT
A percentage	DECIMAL(5,2) with a CHECK between 0 and 100	FLOAT, or INT (0% forever)
A rating 1–5	TINYINT + CHECK	VARCHAR('4 stars')
Currency, country, state codes	CHAR(2) / CHAR(3) with a lookup	VARCHAR(50) free text
A name, label, email	VARCHAR(n) with a meaningful n (254 for email)	CHAR(50), TEXT for everything
Long prose	TEXT/CLOB in a detail table	TEXT in the fact table
An event time	TIMESTAMPTZ / DATETIMEOFFSET / TIMESTAMP (MySQL)	DATETIME, or an unqualified TIMESTAMP
A business date	DATE, computed in the business zone	DATETIME at midnight
A duration	INTERVAL, or BIGINT seconds with a unit in the name	Two timestamps and mental arithmetic
A true/false flag	BOOLEAN (native or BIT/TINYINT(1)) NOT NULL	Nullable flag, CHAR(1) 'Y'/'N'
A status with fixed values	ENUM, or VARCHAR + CHECK	Free text, integer codes with no table
A hash or token	BINARY(32) / BYTEA / RAW(16)	VARCHAR(255) with a case-insensitive collation
Sparse attributes	JSONB / JSON, indexed on the paths you query	One giant NVARCHAR blob
A list you join on	Junction table	Comma-separated string, array
A globally unique key	BIGINT identity + UUID public key	Random UUID alone as a clustered key
A file or image	Object storage + pointer column	BLOB in the hot table

Table 9.1 — The right column versus the tempting column.

Five declarations to memorise

universal · copy, adapt, ship

```
-- Analytics fact table (PostgreSQL)
sale_id BIGINT PRIMARY KEY,
sold_at TIMESTAMPTZ NOT NULL,
business_date DATE NOT NULL,
amount DECIMAL(14,2) NOT NULL,
currency CHAR(3) NOT NULL,
customer_sk INTEGER NOT NULL,
quantity INTEGER NOT NULL CHECK (quantity > 0)

-- Slowly changing dimension
```

```
customer_id VARCHAR(20) NOT NULL UNIQUE,
valid_from TIMESTAMPTZ NOT NULL,
valid_to   TIMESTAMPTZ,                 -- NULL = current row
is_current BOOLEAN NOT NULL DEFAULT TRUE

-- Event log
event_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),
payload JSONB NOT NULL DEFAULT '{}'::jsonb

-- Audit-safe money with minor units (portable, SQLite-friendly)
amount_minor BIGINT NOT NULL,           -- paise
currency     CHAR(3) NOT NULL           -- 'INR'

-- Case-insensitive email uniqueness
email VARCHAR(254) NOT NULL UNIQUE      -- with a _ci collation or a LOWER() index
```

9.2 Fifteen exercises

Each exercise has one clearly best answer, given in 9.3. Try to justify the choice before you read it — the reasoning is the skill, not the type name.

#	Design task
1	A table of Indian PIN codes (6 digits, may start with 0) with the state name.
2	A payment table for a business operating in INR, USD and EUR, reconciled to the cent.
3	A user table where emails must be unique regardless of case, and no email may be missing.
4	An orders table for a marketplace expected to reach 5 billion orders over its lifetime.
5	A global event stream where the same event must never get the same ID on two continents.
6	A store's opening hours: 09:00–21:00 Monday to Saturday, closed Sunday.
7	A subscription with a billing anchor day; reports must show MRR by calendar month in IST.
8	A product catalogue where 40 attributes differ by category and three of them are filtered on.
9	A table of 20 million sensor readings: device, timestamp, numeric value, quality flag.
10	An audit log of API requests, each with a variable JSON response body, retained 90 days.
11	A conference room booking system that must prevent overlapping bookings.
12	A customer dimension with history: address changes must not overwrite the past.
13	A user table for a global app with users in Devanagari, Arabic and Chinese scripts.
14	A report joining a fact table to a dimension, which runs 40× slower after a migration.
15	A hash column for deduplicating uploaded files, compared case-sensitively and exactly.

Table 9.2 — Design tasks, elementary to advanced.

9.3 Answers, with reasoning

1 PIN code must be text

A PIN code is an identifier, not a number: leading zeros exist, arithmetic is meaningless, and sorting by value is meaningless. `pin_code CHAR(6) NOT NULL CHECK (pin_code ~ '^[0-9]{6}$')` plus a state lookup table or a VARCHAR(40).

2 DECIMAL money with a currency column

amount DECIMAL(19,4), currency CHAR(3) CHECK (currency IN ('INR','USD','EUR')), plus an exchange_rate DECIMAL(18,8) and a reporting_amount DECIMAL(19,4). Never FLOAT; never two amounts in one column without the currency.

3 Unique with a case-insensitive collation

email VARCHAR(254) COLLATE utf8mb4_0900_ai_ci NOT NULL UNIQUE (MySQL), CITEXT or a LOWER() unique index (PostgreSQL), NVARCHAR(254) with a _CI collation (SQL Server). Doing the dedupe in application code fails as soon as a second writer exists.

4 BIGINT identity, plus partitioning

INT overflows at 2.1 billion; 5 billion rows is BIGINT territory. order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, and range partition by order date so old partitions can be archived.

5 Time-ordered global IDs

UUIDv7 or ULID (16 bytes, time-ordered), or a Snowflake-style BIGINT (machine id + timestamp + sequence). UUIDv4 works but fragments the index; store it as BINARY(16)/UUID, not CHAR(36).

6 Wall-clock TIME, not an instant

day_of_week TINYINT, opens_at TIME, closes_at TIME. Opening hours are wall-clock values in the store's zone; an instant would shift when DST rules change. Add a zone column if stores span zones.

7 Instant for events, DATE for the accounting period

created_at TIMESTAMPTZ, anchor_day SMALLINT, and a generated business_month DATE computed in IST. MRR by month must group on the IST-derived date, not on the UTC month — otherwise the last 5.5 hours of every month land in the wrong period.

8 JSONB for the long tail, columns for the hot paths

attributes JSONB for the 40, plus STORED generated columns and B-tree indexes for the three filtered attributes, plus a GIN index for containment queries. If a fourth attribute becomes hot, promote it too.

9 Narrow types with a composite primary key

device_id INT, reading_ts TIMESTAMPTZ (or BIGINT epoch if you need compactness), value DECIMAL(10,4) or DOUBLE if the sensor is inherently approximate, is_valid BOOLEAN. Only 4–8 bytes of flag; 20 M rows × indexes is where the cost lives. Partition by month.

10 JSONB in a partitioned table

request_id BIGINT, at TIMESTAMPTZ, status SMALLINT, body JSONB. Retain 90 days by dropping monthly partitions instead of DELETE — the type of the partition key is what makes that possible.

11 Range type with an exclusion constraint

PostgreSQL: during TSTZRANGE plus EXCLUDE USING GIST (room WITH =, during WITH &&). Not achievable declaratively elsewhere: elsewhere you need a unique index on a bucketed slot or row-level locking in the application.

12 Type-2 slowly changing dimension

Surrogate INTEGER key, natural customer_id with a UNIQUE index, valid_from/valid_to TIMESTAMPTZ, is_current BOOLEAN, plus a partial unique index on (customer_id) WHERE is_current to guarantee one current row.

13 Unicode storage chosen once

MySQL: utf8mb4 on the column, table and connection. SQL Server: NVARCHAR(n) (VARCHAR will lose the characters). PostgreSQL/Oracle: UTF-8 database / AL32UTF8. Size in characters but verify the byte budget — 3 bytes per Devanagari character, 4 for emoji.

14 Find the type mismatch, not a faster query

Compare the join columns' types on both sides and look for an implicit conversion (VARCHAR parameter against an INT key, or a different collation). Then check for a function on a filtered column. Add matching types, an expression index if needed, and only then consider partitioning.

15 Fixed-width binary

sha256 BINARY(32) / BYTEA / RAW(32) with a CHECK on length. Binary comparison is byte-exact and case-sensitive, unlike VARCHAR under a case-insensitive collation, and 32 bytes versus 64 characters halves the index.

9.4 Glossary

Term	Meaning
Affinity (SQLite)	The storage class a declared type is converted to; there are only five.
Cardinality	The number of distinct values in a column; drives index and histogram usefulness.
Cast	An explicit conversion between types (CAST, ::, CONVERT).
Collation	Rules for comparing and sorting text: case, accent and language behaviour.
Covering index	An index that contains every column a query needs, so the table is never read.
Domain (PostgreSQL)	A named type with built-in CHECK constraints, reusable across columns.
Implicit conversion	An automatic conversion the engine performs; the usual cause of disabled indexes.
Fixed-width	A type that always occupies its declared size (INT, CHAR, DATE).
Off-page storage	Mechanism that moves large values to overflow pages (TOAST, row-overflow, BLOB pages).
Precision (p)	Total number of significant digits a decimal type can hold.
Scale (s)	Number of digits to the right of the decimal point.
Sargable	A predicate the optimiser can answer with an index seek instead of a scan.
Storage class	SQLite's actual on-disk categories: NULL, INTEGER, REAL, TEXT, BLOB.
SRID	Spatial reference identifier: the coordinate system of a geometry.
Three-valued logic	TRUE, FALSE and UNKNOWN, the last arising from NULL comparisons.
TOAST	PostgreSQL's mechanism for storing large values outside the main row.
Type affinity	See affinity; the reason a DATE column in SQLite can hold text.
Unsigned	A numeric modifier (MySQL) that removes negative values and doubles the positive range.
Variable-width	A type that stores only the bytes needed plus a length prefix (VARCHAR, DECIMAL).
Vector (embedding)	A fixed-length float array used for semantic similarity search.
Widening conversion	A conversion that cannot lose data (INT to BIGINT); the only safe direction.

Table 9.3 — Twenty-one terms that come up in every schema review.

9.5 The last checklist

Before you commit a schema	Why
Every column has the narrowest type that cannot overflow	Storage and index cost are permanent
Money, percentages and counts are exact types	Floating point error is invisible until it is expensive
Every temporal column is explicitly instant or wall clock	The only decision that cannot be fixed later
Every text column declares a charset and a collation	Silent truncation and mojibake start here
Booleans are NOT NULL with a default	Three-valued logic spreads through every consumer
Keys are BIGINT unless proven small; public IDs are separate	Overflow and index fragmentation are years-long problems
Blobs and huge text live outside the hot table	Row width is read cost, even when you do not select it
Structured types are indexed on the paths you query	Otherwise every JSON query is a scan with parsing
A mapping document exists for every migration	The next engineer needs your reasoning, not just your DDL
The schema is reviewed by someone who will run the queries	The person reading rows sees what the designer cannot

Table 9.4 — Ten checks, thirty seconds each.

Closing note

Data types are where intent becomes behaviour. A schema is a set of promises about what the data means, and the database will hold you to those promises long after the person who wrote them has moved teams. That is why this guide spends more time on why than on which: the type names change between engines and versions, but the decisions — exact or approximate, instant or wall clock, bounded or unbounded, one column or many — do not.

If you take one habit from these pages, take this one: before writing a CREATE TABLE, say out loud what each column means and who will never be able to change it. The types follow almost by themselves.

Keep practising Rebuild one of your existing tables with the types this guide recommends and diff the row sizes and query plans. Real schemas teach faster than examples.	**Read the docs by type** Every engine documents each type with storage, range and index notes. Use those pages as the source of truth for version-specific details.
Write it down One page per table: what each column means, why the type was chosen, what cannot change. That document outlives every migration.	**Share it** The fastest way to learn a type system is to review someone else's schema with it by your side.

About this guide

Prepared by **Niraj Laxman Jadhav**, data analyst — Excel, SQL, Power BI and Python — as a practical reference for analysts and engineers who design, review and migrate schemas. Version 1.0, October 2026. Typeset in DejaVu Sans and DejaVu Sans Mono; generated programmatically with ReportLab so every table, page number and contents entry stays consistent.